



ENGINEERING BLUEPRINT · RUNNING EDITION

SentinelEdge

An AI-assisted application, API and edge security platform: architecture, build plan, security design and a step-by-step guide to reproducing it.

SentinelEdge at a glance

SentinelEdge is a security engineering portfolio platform designed to read like a simplified enterprise security product. It is its own first protected workload: every control the platform reports on also protects the platform itself.



What it demonstrates

The project sets out to support one sentence in an interview: *"I can design, build, secure, automate, deploy, monitor, and govern an internet-facing, AI-enabled application in AWS."* It is aimed at Application Security, Product Security, API Security, Cloud Security, DevSecOps, AI Application Security and WAF/CDN Security engineering roles.

- **Secure application engineering:** Argon2id passwords, short-lived JWTs checked against a live session on every request, rotating refresh tokens with theft detection, TOTP MFA, CSRF defenses, and no default credentials anywhere.
- **Authorization that cannot drift:** five roles, an access rule declared on every route, and a test that sweeps every route as every role and fails if code and policy disagree.
- **Tamper-evident audit:** a SHA-256 hash chain, append-only by database grant *and* trigger, verifiable from the CLI, the API and the UI.
- **API security:** atomic PostgreSQL token-bucket rate limiting, spoof-proof client-IP resolution, and a live API inventory generated from the route table. OWASP API Top 10 coverage is mapped, with cited test evidence.
- **Honesty about what is real:** every capability is labelled REAL_AWS, LOCAL, SIMULATED or DEMO, and tests stop anything being presented as an AWS control before it is one.
- **Cloud architecture designed ahead of spend:** a private-origin AWS design (CloudFront VPC origins, internal ALB, isolated RDS, WAF changes only through Terraform), built late on purpose so no infrastructure sits idle.

How to read this book

Recruiters and hiring managers: this page, Chapter 1 and Chapter 2 tell the story in about ten minutes. **Engineers and reviewers:** Part I is the blueprint (architecture, stack and security design). Part II walks through each completed phase: what was built, why, how it works and how it is verified. Part III describes the remaining phases as designed. **Anyone reproducing the build:** Part IV contains the commands, in order, that bring the platform up from an empty machine and verify it.

About this edition

This is a *running* book. A new edition is produced with every phase release, and each completed phase moves from Part III (planned) into Part II (as built). Edition 1 reflects release **v0.3.0**, with Phases 1, 2 and 6 complete. Everything described as built has been run and tested. Everything planned is labelled as planned. All data in the platform is synthetic.

Contents

PART I — THE BLUEPRINT

Chapter 1	Introduction	8
	1.1 What SentinelEdge is	8
	1.2 The first protected workload	8
	1.3 Design principles.....	8
	1.4 Who it is for	9
	1.5 Current status.....	9
Chapter 2	The Build Plan	10
	2.1 The twelve phases	10
	2.2 Build order: local first	11
	2.3 Definition of done.....	11
	2.4 Delivery workflow	12
	2.5 Releases so far	13
Chapter 3	Architecture Blueprint	14
	3.1 Target architecture on AWS	14
	3.2 Why each integration sits where it does	15
	3.3 Local topology today.....	16
	3.4 Application structure.....	17
	3.5 Data model	19
	3.6 Environments.....	19
Chapter 4	Technology Stack and Rationale	20
	4.1 Backend.....	20
	4.2 Database	20
	4.3 Frontend	21
	4.4 Infrastructure and delivery	21
	4.5 Development environment.....	21
Chapter 5	Security Architecture	22
	5.1 Defense in depth: seventeen layers	22
	5.2 Threat model	23
	5.3 The control matrix	24
	5.4 Real, local, simulated, demo	24
	5.5 Governed exceptions	25

PART II — AS BUILT

Chapter 6	Phase 1 — Architecture and Secure Foundation	27
	6.1 Goals.....	27
	6.2 Repository layout	27
	6.3 Decisions recorded up front	27
	6.4 Cross-cutting mechanisms.....	28
	6.5 Hardened containers.....	29
	6.6 Shift-left gates (ADR-0013).....	29
	6.7 Talking points	30

Chapter 7	Phase 2 — Secure Application Foundation	31
	7.1 Milestones	31
	7.2 Database least privilege (ADR-0015)	31
	7.3 Authentication (ADR-0003)	32
	7.4 Authorization	33
	7.5 Tamper-evident audit log (ADR-0005)	34
	7.6 Frontend	34
Chapter 8	Phase 6 — API Security	36
	8.1 Milestones	36
	8.2 One registry for every endpoint.....	36
	8.3 Rate limiting (ADR-0017)	36
	8.4 Trusting the right client IP	38
	8.5 Route templates	38
	8.6 Endpoint metrics	39
	8.7 The API Security Center	39
	8.8 OWASP API Security Top 10 (2023)	39
	8.9 Route-table sweeps	40
	8.10 SSRF guard.....	40
PART III — THE ROAD AHEAD		
Chapter 9	Phase 7 — Security Operations	42
	9.1 Scope	42
	9.2 Design commitments already made	42
	9.3 Security focus	43
Chapter 10	Phase 8 — Application Security Scanning	44
	10.1 Scope	44
	10.2 Vulnerability management	44
Chapter 11	Phase 10 — Threat Modeling and Governance	45
	11.1 Scope	45
Chapter 12	Phase 9 — AI Security Engine	46
	12.1 Scope	46
	12.2 The output contract (ADR-0007)	46
Chapter 13	Phases 3–5 — AWS Foundation, Deployment and Edge	47
	13.1 Phase 3 — Terraform AWS foundation.....	47
	13.2 Phase 4 — Application deployment	47
	13.3 Phase 5 — CloudFront, WAF and TLS	48
Chapter 14	Phases 11–12 — Automation and Final Hardening	49
	14.1 Phase 11 — Automation and DevSecOps.....	49
	14.2 Phase 12 — Portfolio hardening.....	49
	14.3 Demo mode	49
	14.4 Acceptance criteria.....	49
PART IV — REPRODUCING THE BUILD		
Chapter 15	Reproducing the Build	52

15.1	Prerequisites	52
15.2	Step 1 — Prepare the machine.....	52
15.3	Step 2 — Get the source	52
15.4	Step 3 — Create the Python environment and install dependencies	53
15.5	Step 4 — Generate local secrets	53
15.6	Step 5 — Start the stack	53
15.7	Step 6 — Create the first administrator.....	53
15.8	Step 7 — Prove the controls	53
15.9	Step 8 — Ship a phase the way this project does.....	54
15.10	Day-to-day reference.....	55
15.11	Troubleshooting	55
Chapter 16	Testing and Evidence	56
16.1	The layers	56
16.2	Negative tests that matter	56
16.3	Tests that test the tests	57
16.4	Continuous integration.....	57
Chapter 17	Lessons Learned So Far	58
17.1	Engineering	58
17.2	Testing and tooling.....	58
17.3	Process.....	59
Appendix A	Architecture Decision Records	60
Appendix B	Threat Register	61
Appendix C	Implemented Controls.....	63
Appendix D	API Endpoint Inventory	65
Appendix E	Glossary and Edition History	66
E.1	Glossary	66
E.2	Edition history.....	66

Part I — The Blueprint

What SentinelEdge is, how it is being built, and the architecture, technology and security design that every phase builds on.

CHAPTER 1

Introduction

Most security portfolios show a tool. SentinelEdge shows a system: an internet-facing, AI-assisted application whose design, code, infrastructure, pipeline and governance are all built to be defended in an interview, line by line.

1.1 What SentinelEdge is

SentinelEdge is an application, API and edge security platform. When complete, it gives a security team one place to see and act on the security of a web application:

- edge and WAF activity, with rule changes governed through Terraform;
- an inventory of every API endpoint, its controls and its traffic;
- security events, incidents and their investigation from detection to closure;
- vulnerability findings from SAST, SCA, DAST, IaC and container scanning, and SBOMs;
- AI-assisted analysis of events through Amazon Bedrock, with prompt-injection defenses and human approval;
- threat models, a control matrix, risk exceptions with expiry, and change records.

It deliberately resembles a simplified enterprise security platform. It is not a generic SaaS application and it is not an AI chatbot. The application itself has to demonstrate security engineering competence.

1.2 The first protected workload

The design rests on one idea: **SentinelEdge protects itself first**. The WAF it reports on sits in front of SentinelEdge. The rate limits its API Security Center displays are the limits enforced on its own API. The audit log it verifies records its own administrators' actions. The attack simulator targets only SentinelEdge, never an external system.

This removes the usual gap between a demo and reality. Every dashboard figure is backed by a control actually running on the platform. Where a control cannot run yet (AWS WAF before the AWS phases, say), the platform says so explicitly rather than simulating it silently.

1.3 Design principles

1. **Defense in depth.** Seventeen layers, from DNS to AI controls, each with a stated reason to exist (Chapter 5). No single control is load-bearing.
2. **Secure by default.** No default credentials, interactive API docs off when deployed, insecure configuration refused at startup, every container non-root and read-only.
3. **Evidence over assertion.** Every claimed control has a negative test that proves the attack fails, and the documentation points to it. An OWASP mapping that cites a test fails CI if that test disappears.

4. **Real versus simulated, enforced in code.** Every capability carries a provenance label (REAL_AWS, LOCAL, SIMULATED, DEMO). Tests stop a simulation being presented as an AWS control.
5. **Decisions are written down.** Every decision with security trade-offs gets an Architecture Decision Record (ADR) covering context, decision, security impact, alternatives and consequences. Seventeen exist so far.
6. **Build incrementally, stop for approval.** Twelve phases, each ending in a tested, documented, tagged release and a deliberate pause before the next.
7. **Spend nothing until it buys something.** All local phases are built before any AWS resource exists, so cloud infrastructure runs for one focused window instead of idling for months.

1.4 Who it is for

The project is built to support conversations for these roles:

Role	Where the evidence is
Application / Product Security Engineer	Authentication, RBAC, audit log, secure error handling (Chapters 6–7)
API Security Engineer	Rate limiting, inventory, OWASP API Top 10 coverage (Chapter 8)
Cloud Security Engineer	Private-origin AWS design, IAM, state isolation (Chapters 3, 13)
WAF / CDN Security Engineer	CloudFront, WAF rule governance via Terraform (Chapters 3, 13)
DevSecOps Engineer	Shift-left gates from Phase 1, SBOM, OIDC pipeline (Chapters 6, 10, 14)
AI Application Security Engineer	Bedrock integration, output contract, human approval (Chapter 12)
Security Operations	Events, incidents, runbooks, attack simulator (Chapter 9)

1.5 Current status

Phase	Scope	Status
1	Architecture, repository, ADRs, local environment, CI baseline	COMPLETE · V0.1.0
2	Secure application foundation: auth, MFA, RBAC, audit logging	COMPLETE · V0.2.0
6	API security: inventory, OWASP API mapping, rate limiting	COMPLETE · V0.3.0
7	Security operations: events, dashboard, incidents, simulator	NEXT
8, 10, 9	AppSec scanning · threat modeling and governance · AI security	PLANNED
3, 4, 5	AWS foundation, deployment, CloudFront + WAF + TLS	PLANNED
11, 12	Automation pipeline · hardening and final review	PLANNED

Phase numbers come from the original specification and keep their meaning everywhere. The *build order* differs from the numbering on purpose; Chapter 2 explains why.

CHAPTER 2

The Build Plan

SentinelEdge is built from a 60-section specification, in twelve phases. Each phase ends in a tested release and a deliberate stop for approval. This chapter explains the phases, the order they are built in, and the delivery workflow that keeps every step recoverable.

2.1 The twelve phases

The specification divides the work into twelve phases, each with a fixed scope:

Phase	Name	Scope
1	Architecture & Repository	Repository, directory structure, architecture docs, ADRs, Docker environment, README
2	Secure Application Foundation	FastAPI, React, PostgreSQL, authentication, RBAC, API foundation, secure and audit logging
3	Terraform AWS Foundation	VPC, subnets, routing, security groups, IAM, ECR, Terraform state architecture
4	AWS Application Deployment	ECS Fargate, ALB, RDS, Secrets Manager, CloudWatch
5	CloudFront + WAF + TLS	Route 53, CloudFront, ACM, AWS WAF, HTTPS, edge security headers
6	API Security	API inventory, API security controls, rate limiting, authorization, OWASP API mappings
7	Security Operations	Event collection, security dashboard, incident management, WAF event visualization, HTTP analysis
8	Application Security	SAST, SCA, DAST, secrets, IaC and container scanning, SBOM
9	AI Security	AI security engine, prompt defense, analysis, AI risk scoring, AI audit logging, human approval
10	Threat Modeling & Governance	STRIDE, PASTA, controls, exceptions, risk acceptance, change management
11	Automation & DevSecOps	GitHub Actions, Terraform validation, security gates, automated reports, automation scripts
12	Portfolio Hardening	Full security review, dependency/IAM/Terraform/WAF/API/AI reviews, demo scenarios, final documents

Within every phase the same twelve-step discipline applies: explain the architecture, explain the security decisions, create the files, implement working functionality, implement automated tests, implement security controls, run validation, fix errors, update documentation, explain what was completed, identify the next phase, and **stop**.

2.2 Build order: local first

Followed in numeric order, the plan would stand up AWS infrastructure at Phases 3–5 and then build seven more phases of mostly local features on top of it. An always-on development environment (NAT gateway, ALB, RDS, WAF, Fargate) costs roughly **\$110–125 per month**, so months of feature work would pay for idle infrastructure.

ADR-0016 reorders the build without changing its scope:

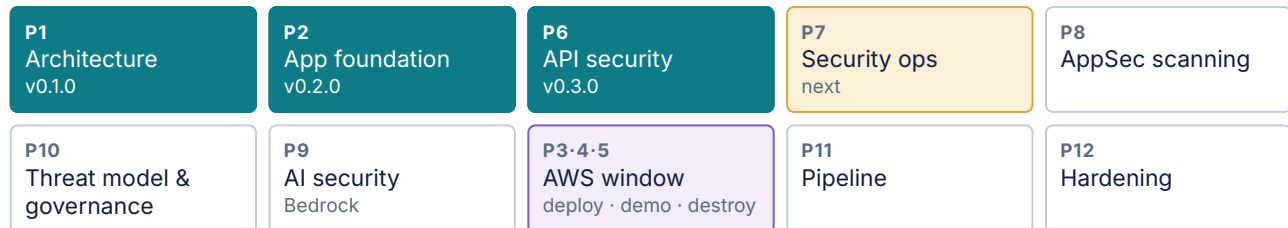


Figure 1. Build order, left to right and top to bottom. Teal: complete. Amber: next. Violet: the AWS deployment window.

Order	Phase	Runs on	AWS cost while building
1–3	1, 2, 6	Local	\$0 (done)
4–6	7, 8, 10	Local (+ CI)	\$0
7	9 AI security	Local + Amazon Bedrock API calls	Cents
8–10	3, 4, 5	AWS, one focused deployment window	Days, not months
11–12	11, 12	CI + AWS	Short

Three rules keep this honest:

- **Phase numbers keep their original meaning** in documentation, the capability register and commits, so the specification's acceptance criteria map cleanly.
- **Terraform is written and statically scanned before anything is applied.** `terraform validate` and Checkov cost nothing.
- **Features that will read live AWS data are built first against labelled SIMULATED or DEMO sources.** They switch to REAL_AWS in the AWS phases, and the provenance rules (ADR-0009) make that switch explicit and tested.

After the AWS phases, evidence (plan/apply output, scanner reports, WAF blocks, origin-unreachable proof, a walkthrough video) is captured into `docs/evidence/`. The environment is then destroyed and re-created on demand for interviews.

2.3 Definition of done

A phase is complete only when every item that applies to it is true. This list lives in `docs/secure-sdlc.md`:

1. The threat model is updated for any new boundary, data flow, asset or role.
2. Controls are recorded in the control matrix with implementation and evidence.
3. The capability register is updated with correct provenance, and its guard tests are adjusted deliberately.
4. Negative tests exist for every new control (the attack fails), not only positive tests.
5. All gates pass: Gitleaks, Ruff, Bandit, mypy strict, pip-audit, npm audit, ESLint, tests, coverage $\geq 90\%$.

6. No secrets are in code, images, Terraform or workflow files.
7. An ADR is written for any decision with security trade-offs.
8. Documentation for the phase is written, and the README status table is updated.
9. Real-versus-simulated wording is checked in the UI and docs.
10. A runbook is added or updated for any new operational failure mode.

2.4 Delivery workflow

Every phase moves through the same pipeline, so that any point in the project's history can be recovered:

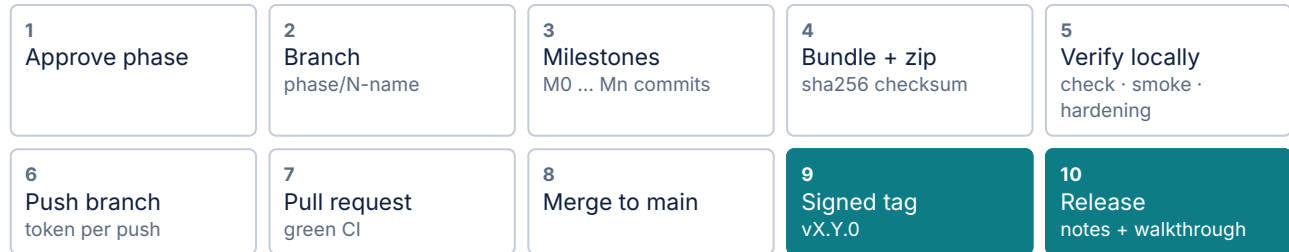


Figure 2. Phase delivery workflow, from approval to a published GitHub release.

- **Branch per phase.** Work lands on `phase/N-name` (for example `phase/6-api-security`), never directly on `main`.
- **Milestones.** Each phase is cut into milestones (M0, M1, ...). Each is a single commit that leaves the system working and tested. M0 is always housekeeping. The last milestone is documentation, the smoke test and release notes.
- **Verifiable delivery.** Milestones are delivered as a git bundle inside a zip, with a published SHA-256 checksum. The bundle is verified (`git bundle verify`) and fetched into the local repository, so history arrives exactly as built.
- **Local verification before push.** `make check` (every CI gate), `make smoke` (end-to-end) and `make verify-hardening` (container and network controls) must pass on the owner's machine.
- **Push, PR, CI.** The branch is pushed with a short-lived fine-grained GitHub token, pasted at push time and never stored. A pull request runs the same gates in GitHub Actions.
- **Merge, sign, release.** After a green merge, the release is tagged with an SSH-signed annotated tag (`vX.Y.0`) and published as a GitHub release with notes from `docs/releases/`.
- **Walkthrough.** Every phase ships with a step-by-step walkthrough document. It explains each step, why it comes where it does in the workflow, and the commands that produced the intended result.

WHY BUNDLES INSTEAD OF PATCHES

A git bundle carries commits with their IDs, authorship and parents intact. Verifying it confirms that the pre-requisite commit is present before anything is applied. The zip and checksum guard the transfer. The result is that the repository on GitHub is byte-for-byte the history that was built and tested.

2.5 Releases so far

Release	Phase	Milestones	Headline
v0.1.0	1	single commit	Architecture, secure foundation, local environment, CI baseline
v0.2.0	2	M0–M7	Authentication with MFA, RBAC, tamper-evident audit log, least-privilege database
v0.3.0	6	M0–M5	Rate limiting, trusted client IPs, API Security Center, OWASP API Top 10 coverage

CHAPTER 3

Architecture Blueprint

This chapter describes the target architecture on AWS, the local topology that stands in for it today, and how the application itself is structured. Each integration sits where it does for a security reason, and that reason is stated alongside it.

3.1 Target architecture on AWS

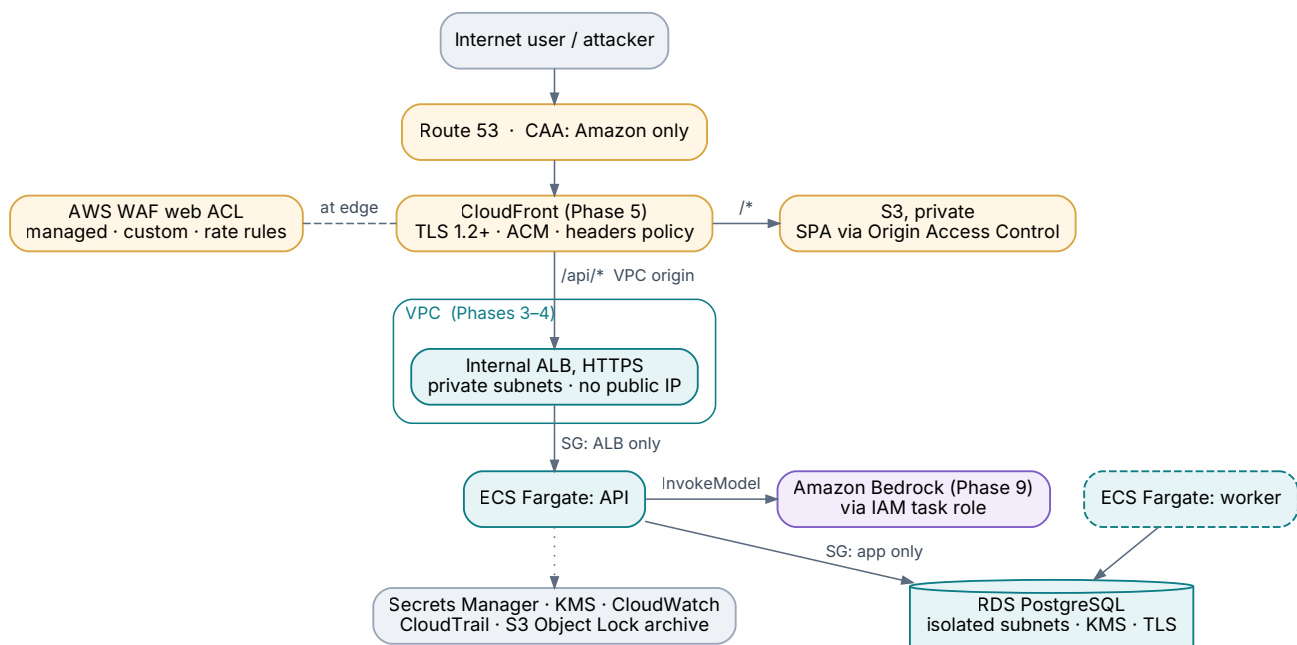


Figure 3. Target AWS architecture. Amber: edge (Phase 5). Teal: VPC workloads (Phases 3–4). Violet: AI (Phase 9).

A request travels through these tiers:

1. **Route 53** resolves the domain. A CAA record allows only Amazon to issue certificates for it, so a mis-issued certificate from another CA is refused at issuance.
2. **CloudFront** terminates TLS (TLS 1.2+ with an ACM certificate) close to the user and applies a response-headers policy.
3. **AWS WAF** is attached to the distribution and evaluated *at the edge, before any origin request*. It applies managed rule groups, custom rules and rate-based rules.
4. CloudFront routes by path. `/*` goes to a **private S3 bucket** holding the SPA, reached through Origin Access Control. `/api/*` goes through a **CloudFront VPC origin** to an **internal Application Load Balancer**.
5. The ALB forwards to **ECS Fargate** tasks in private subnets with no public IP addresses.

6. The tasks reach **RDS PostgreSQL** in isolated subnets with no route to the internet, encrypted with KMS and requiring TLS.
7. The API calls **Amazon Bedrock** with its IAM task role, reads secrets from **Secrets Manager**, and logs to **CloudWatch**. CloudTrail records API activity. Audit records are archived to S3 with Object Lock.

Security groups chain the tiers. The ALB admits only the CloudFront VPC-origin service security group. The application admits only the ALB on port 8000. The database admits only the application on port 5432.

A CORRECTION TO THE SPECIFICATION'S DIAGRAM

The specification draws WAF as a network hop between CloudFront and the ALB. In AWS, WAF is attached to the CloudFront distribution and evaluated at the edge, before the origin request is made. The security outcome is the one the specification intends, but the documentation describes the real mechanism, because an interviewer will know the difference.

3.2 Why each integration sits where it does

No public origin (ADR-0001)

If the load balancer were internet-reachable, an attacker who found it could bypass CloudFront and the WAF entirely (threat T-EDGE-03). The common mitigations are an ALB security group limited to the CloudFront prefix list, plus a secret header. They reduce the risk but leave a public endpoint, and the prefix list admits traffic from *any* CloudFront distribution, including an attacker's. Making the ALB **internal** and connecting it with **CloudFront VPC origins** removes the bypass path instead of mitigating it. The trade-off is that tasks need NAT or VPC endpoints for egress.

One origin, no CORS (ADR-0002)

The SPA and the API share one hostname. CloudFront (locally, nginx) routes `/api/*` to the API and everything else to the SPA. There is therefore no CORS policy to misconfigure, and the API sends no `Access-Control-Allow-*` headers at all. The refresh cookie can be `SameSite=Strict` and host-only, and one WAF web ACL and one headers policy cover all traffic. The SPA's API client refuses absolute and protocol-relative URLs, so it cannot be tricked into calling another origin.

WAF changes only through Terraform (ADR-0008)

A dashboard holding `wafv2:Update*` permissions would be a high-value target and would make Terraform state drift from reality. The application gets a **read-only** WAF role. A requested change becomes a change request. Once approved, it is applied by a pull request that updates Terraform variables, and CI plans and applies it. In-dashboard rule toggling exists only in the simulator and is labelled SIMULATED. Real changes take minutes, not seconds. That is how mature teams operate.

Amazon Bedrock via IAM role (ADR-0006)

The AI engine uses Amazon Bedrock (Anthropic Claude models) behind a provider interface. The ECS task role is granted `bedrock:InvokeModel` on specific model ARNs only. No API key exists, so there is no AI secret to store or leak. Prompts can stay off the public internet through a Bedrock runtime VPC endpoint, and CloudTrail records every invocation. Locally an `offline` provider gives deterministic results for tests, and configuration validation rejects it in production.

Security headers at two layers (ADR-0011)

Headers are set by the API middleware (strictest CSP: `default-src 'none'`) and by the edge (nginx locally, a CloudFront response-headers policy on AWS) with the SPA's policy. If the edge is misconfigured or bypassed, the application still protects itself. If the application regresses, the edge still applies headers.

Tamper-evident audit with an off-host anchor (ADR-0005)

Audit records are hash-chained and append-only inside PostgreSQL. A chain proves that what remains is unaltered, but not that nothing was cut from the end. Periodic export of the chain head to S3 with Object Lock (Phase 4) closes that gap, so the anchor sits outside the database on purpose.

3.3 Local topology today

Until the AWS phases, Docker Compose reproduces the same trust boundaries on one machine:

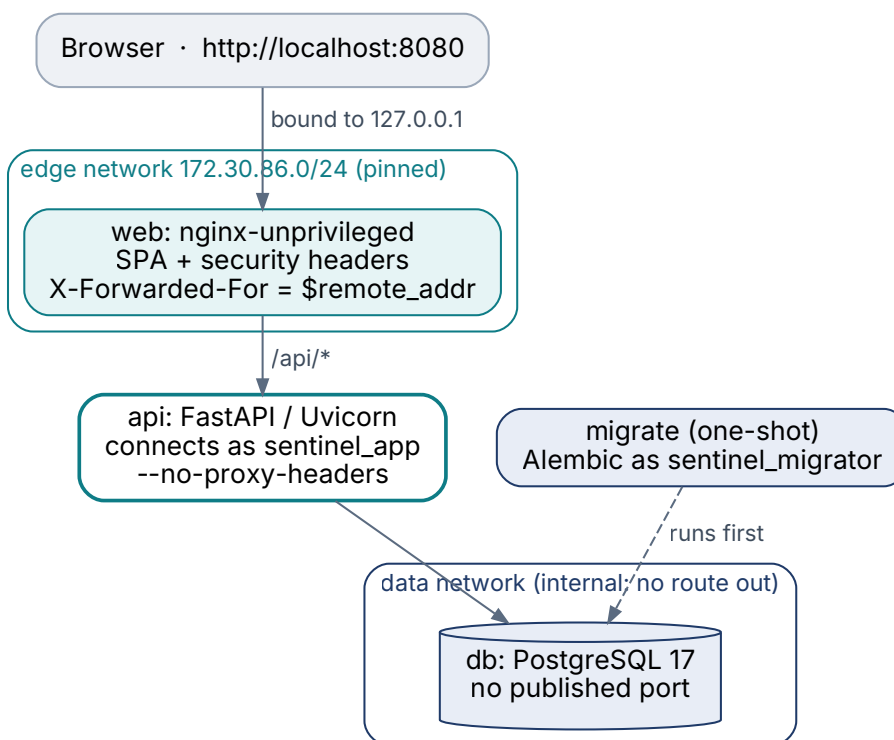


Figure 4. Local topology. The web container stands in for CloudFront; the internal data network stands in for isolated subnets.

Concern	Local (now)	AWS (planned)
Edge and routing	nginx <code>web</code> container	CloudFront (Phase 5)
WAF	none (labelled as such)	AWS WAF (Phase 5)
SPA hosting	nginx	S3 + Origin Access Control
API	<code>api</code> container	ECS Fargate (Phase 4)
Database isolation	Docker <code>internal: true</code> network, no published port	Isolated subnets + security group
Secrets	<code>.env</code> , generated, git-ignored, mode 600, scoped per container	Secrets Manager
Logs	JSON to stdout	CloudWatch Logs
Client IP trust	Pinned edge subnet <code>172.30.86.0/24</code>	ALB subnet ranges

Every container is non-root, has a read-only root filesystem, drops all Linux capabilities and runs with `no-new-privileges`. Ports are bound to `127.0.0.1` only, and the database has no published port at all. Startup order is enforced: `db` initializes and runs the role bootstrap script, `migrate` applies Alembic migrations as `sentinel_migrator` and exits, `api` starts as `sentinel_app`, then `web`.

3.4 Application structure

Backend layering (ADR-0012)

```

backend/app/
  api/v1/      routers; authentication and authorization dependencies
  core/       config, logging, correlation IDs, middleware, errors, provenance,
              capability register, authz, rate limiting, API policy registry
  security/   passwords (Argon2id), tokens (JWT), MFA (TOTP), rate limiter, egress guard
  services/   business rules: auth, users, audit, API inventory
  models/     SQLAlchemy ORM models
  schemas/    Pydantic request and response models
  db/         engine, sessions, migration helpers
alembic/versions/ 0001 baseline ... 0005 api_endpoint_stats

```

Routers call services, services call the ORM, and routers return **Pydantic response models, never ORM objects**. Request models use `extra="forbid"`. These two rules prevent excessive data exposure (a field is returned only if it is declared) and mass assignment (an undeclared field is rejected) structurally, rather than by remembering to check. Directories are created only when their first real code lands, so the tree never contains empty placeholders presented as features.

Request pipeline

Every API request passes the same ordered set of controls:

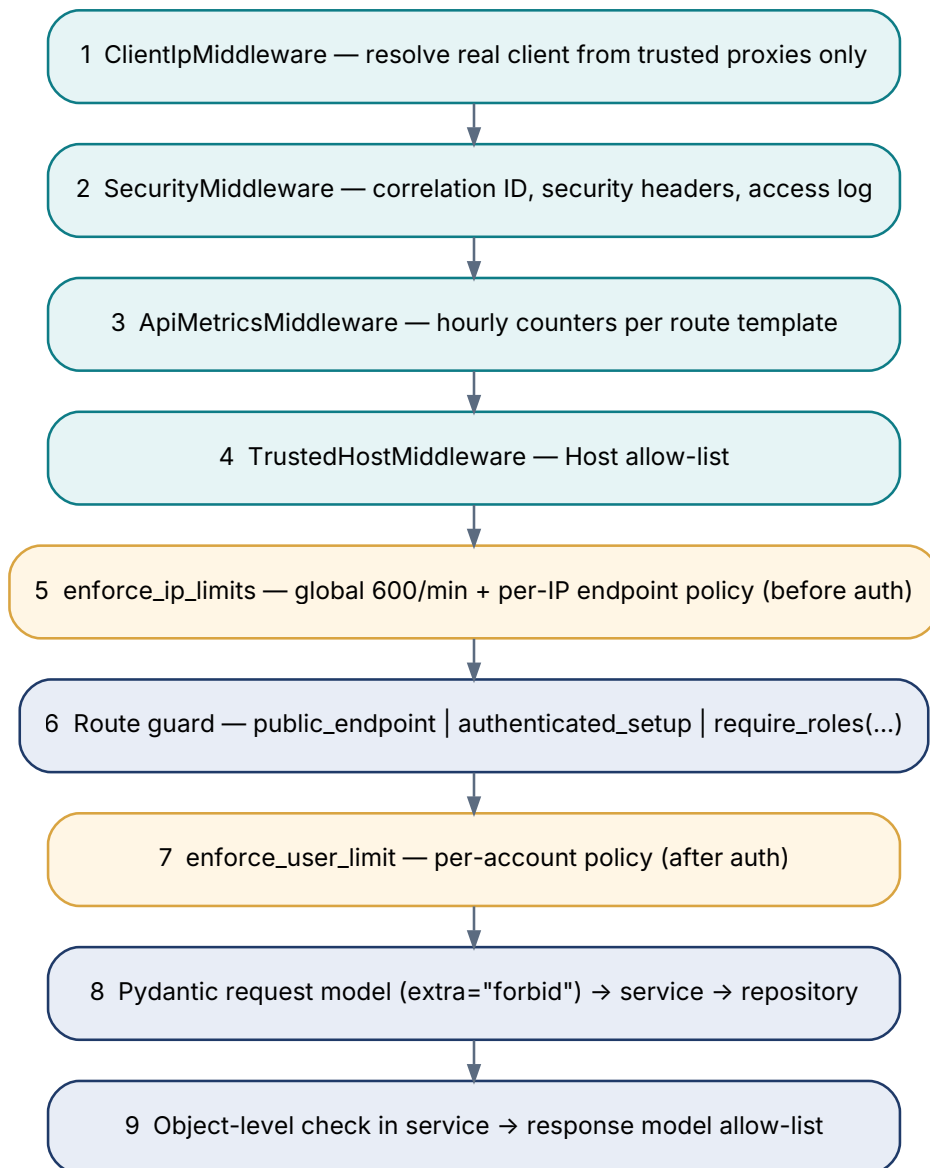


Figure 5. The request pipeline. Middleware (teal) runs on every request; rate limits (amber) sit before and after authentication.

The order matters. Client-IP resolution runs outermost, so every later consumer (rate limiting, audit, access logs) sees the same resolved address. Per-IP limits run *before* authentication, so a flood is rejected before any password hashing. Per-account limits run *after* it, so rotating IP addresses does not reset them.

Frontend

The SPA is React 19 with strict TypeScript, built by Vite and styled with Tailwind CSS. It calls only relative `/api/v1/...` paths through a client that refuses cross-origin URLs and redirects and validates every response's shape. The access token is held **in memory only**, never in web storage (an ESLint rule bans web storage outright). A page reload performs a silent refresh using the HttpOnly cookie, single-flight within a tab and serialized across tabs with the Web Locks API.

3.5 Data model

Primary keys are UUIDs, so record IDs are not enumerable. Timestamps are UTC. Every telemetry table carries a non-null `provenance` column.

Domain	Entities	Phase
Identity	users (MFA secret encrypted in-row), auth_sessions, refresh_tokens (hashed), mfa_recovery_codes (hashed), password_reset_tokens (hashed), outbox_messages	2 ✓
Audit	audit_log (hash-chained, append-only by grant and trigger)	2 ✓
API security	rate_limit_buckets, api_endpoint_stats (hourly, route templates only)	6 ✓
Inventory	applications	7
Telemetry and operations	security_events, incidents, incident_timeline, analyst_notes	7
Edge and WAF	waf_rules (mirror), waf_exceptions, ip_lists	5–7
Vulnerabilities and supply chain	vulnerabilities, scan_runs, sbom_documents, sbom_components	8
Certificates	certificates	5
AI	ai_analyses, ai_action_proposals	9
Governance and posture	threat_models, threats, controls, control_mappings, risk_exceptions, change_requests, posture_snapshots	10

3.6 Environments

`local` and `test` run on a workstation or CI runner. `dev`, `staging` and `production` are AWS environments, each with its own Terraform root and state (ADR-0014). Configuration validation treats the AWS environments as *deployed*. In those, API docs are forced off, DEBUG logging is refused, rate limiting cannot be disabled, database TLS must be verified (`verify-ca` or `verify-full`), and the offline AI provider is rejected. Unknown environment names are refused outright.

CHAPTER 4

Technology Stack and Rationale

Each technology was chosen for a reason that can be explained in a sentence, and a rejected alternative is recorded next to it. Versions are pinned: Python dependencies by hash, npm by lockfile, CI actions by commit SHA.

4.1 Backend

Technology	Version	Why it was chosen	Alternative considered
Python	3.12	Mature security tooling (Bandit, pip-audit, mypy) and a strong typing story	—
FastAPI	0.142	Dependency injection makes authorization a declared, testable property of every route	Flask (authorization by convention, harder to sweep)
Pydantic	v2	<code>extra="forbid"</code> request models and explicit response models enforce API3 structurally	Hand-written validation
SQLAlchemy	2.1	Typed ORM; parameterized queries by construction	Raw SQL
Alembic	1.20	Versioned migrations with an <code>alembic check</code> drift gate in CI	Auto-create tables (no history, no grants)
psycopg	3	Modern PostgreSQL driver with TLS verification support	psycopg2
Uvicorn	0.54	ASGI server; proxy-header handling disabled so the app makes the only trust decision	—
argon2-cffi	25.1	Argon2id: memory-hard, the current OWASP recommendation	bcrypt, PBKDF2
PyJWT	2.15	Algorithm pinned server-side; <code>aud</code> , <code>iss</code> and <code>exp</code> required	—
pyotp + cryptography	—	TOTP MFA; secrets Fernet-encrypted with a key outside the database	SMS (SIM-swap risk)

4.2 Database

PostgreSQL 17 (local container, RDS on AWS). It does more than store data here. Its `INSERT ... ON CONFLICT` makes the rate limiter atomic across API instances without Redis. Its advisory locks serialize the audit hash chain. Its grants and triggers enforce append-only audit history. Three roles separate duties: an admin that bootstraps, `sentinel_migrator` that owns the schema, and `sentinel_app` with only per-table grants (ADR-0015).

WHY NOT REDIS OR ELASTICACHE FOR RATE LIMITING?

ElastiCache would add roughly \$12 or more per month and another service to secure. PostgreSQL's single-statement upsert gives a correct shared counter at portfolio scale. The cost is one database write per limited request, which is recorded as a known consequence and scheduled for review in Phase 12.

4.3 Frontend

Technology	Version	Why
React	19	Escapes output by default; component model fits an enterprise console
TypeScript	6, strict + <code>exactOptionalPropertyTypes</code>	Response shapes are validated and typed end to end
Vite	8	Fast builds with hashed asset names, which enables <code>immutable</code> caching
Tailwind CSS	4	Styles compile to a static stylesheet, so CSP can forbid inline styles
Vitest + Testing Library	5	Behavior-level UI tests, including authorization-aware rendering
ESLint + <code>jsx-a11y</code>	9	Security rules ban <code>dangerouslySetInnerHTML</code> , <code>innerHTML</code> , <code>eval</code> and web storage; accessibility rules are a quality gate
nginx-unprivileged	—	Serves the SPA as non-root with strict headers; local stand-in for CloudFront

4.4 Infrastructure and delivery

Technology	Role	Phase
Docker Compose	Local topology with hardened containers and isolated networks	1
GitHub Actions	CI gates, read-only token, SHA-pinned actions	1 (baseline), 8, 11
Gitleaks, Ruff (S rules), Bandit, mypy, pip-audit, npm audit	Shift-left gates in pre-commit and CI	1
Semgrep, Checkov, Trivy, Syft, OWASP ZAP	Full scanning and SBOM	8
Terraform ≥ 1.10	All AWS infrastructure; S3 state with native locking	3
AWS: CloudFront, WAF, ACM, Route 53, ECS Fargate, ALB, RDS, Secrets Manager, KMS, CloudWatch, CloudTrail	Production architecture	3–5
Amazon Bedrock	AI analysis via IAM task role	9
GitHub OIDC	No long-lived AWS keys in CI	11

4.5 Development environment

The project is developed on Garuda Linux (Arch-based) with the fish shell. All project entry points are `make` targets, so they behave the same in any shell. The few commands that differ in fish are noted in Part IV, for example `source .venv/bin/activate.fish` and `read -s` for a token prompt. Python tooling uses a `uv`-managed 3.12 virtual environment that matches CI.

CHAPTER 5

Security Architecture

This chapter covers the security design behind the build: seventeen defense-in-depth layers, a STRIDE threat model per trust boundary, a control matrix that links each requirement to its evidence, and the rules that keep the platform's claims honest.

5.1 Defense in depth: seventeen layers

#	Layer	Why it exists	Phase
1	DNS	Records managed as code; CAA limits who can issue certificates	5
2	CDN / edge	Terminates TLS close to users, hides the origin, absorbs volume	5
3	WAF	Blocks known attack patterns before they cost origin resources	5
4	Load balancer	Private origin, HTTPS, desync protection	4
5	Security groups	Each tier reachable only from the tier above it	3
6	Private networking	Application and database never directly routable from the internet	3 (local analogue: 1)
7	Authentication	Proves identity; resists credential stuffing and token theft	2 ✓
8	API authorization	Stops BOLA and BFLA regardless of what the UI shows	2, 6 ✓
9	Application validation	Rejects malformed and over-posted input; limits output	1, 2, 6 ✓
10	Database security	Least-privilege roles, encryption, parameterization	2 ✓, 4
11	IAM	Least privilege per function, no static keys	3, 11
12	Secrets management	Secrets never in code, images, Terraform or CI configuration	1 ✓ (scan), 4
13	Logging	Traceability and forensics	1, 2 ✓
14	Monitoring	Detects abuse and failures	4, 7
15	Vulnerability management	Findings tracked to closure with SLAs	8
16	CI/CD security	Stops vulnerable or secret-bearing code shipping	1 ✓, 8, 11
17	AI security	Contains prompt injection and AI agency	9

The test of each layer is simple: if the layer above it fails, does this one still hold? For example, injection is stopped by WAF SQLi rules at the edge, by validated request models, and by ORM-only parameterized queries, so it is still stopped if the WAF is bypassed. Credential stuffing meets a WAF rate rule, per-IP limits, per-account lockout and then MFA.

5.2 Threat model

The method is **STRIDE per trust boundary**. The OWASP Top 10 (2021), OWASP API Security Top 10 (2023) and OWASP Top 10 for LLM Applications (2025) serve as threat catalogues. Risk is likelihood (1–3) × impact (1–3). Each threat carries a status: *Mitigated* (control implemented and tested), *Planned (phase)*, or *Accepted (interim)* with an expiry. The model is reviewed at the end of every phase and whenever a boundary, data flow or role changes. It is at version 0.3. A full PASTA treatment and in-app modeling arrive in Phase 10.

Assets

User credentials, MFA secrets and refresh tokens; security telemetry; the audit log; WAF and edge configuration; AWS credentials and IAM roles; secrets (database credentials, JWT signing key); source, pipeline and images; AI prompts and outputs; Terraform state.

Trust boundaries

Boundary	Crossing	Example threats
TB1	Internet → edge	Floods, TLS downgrade, cache poisoning, Host header injection
TB2	Edge → origin	Origin bypass, spoofed X-Forwarded-For, request smuggling
TB3	Browser session → API	Credential stuffing, token theft, CSRF, enumeration, BOLA, BFLA, mass assignment
TB4	API → database	Exposure, credential theft, over-privileged role
TB5	API → AI provider	Direct and indirect prompt injection, insecure output handling, excessive agency
TB6	CI/CD → AWS	Stolen cloud keys, hijacked third-party actions
TB7	Operators → platform	WAF weakened without review, audit tampering
TB8	Developer → repository	Committed secrets, vulnerable dependencies

Top attack paths and where they break

1. **Origin bypass, then unfiltered injection.** Broken by the private origin (ADR-0001). Application-layer validation and parameterization would still hold even without the WAF.
2. **Credential stuffing, then session theft, then data access.** Broken by the WAF rate rule, per-IP limits, account lockout, MFA, 15-minute tokens checked against a live session, and RBAC with object checks. Detectable through audited failures and `ratelimit.exceeded` records.
3. **A poisoned log line leads the AI to recommend a harmful change, and an operator applies it.** Broken because AI output is schema-bound and advisory, proposals need human approval, and real WAF changes need a reviewed Terraform pull request.

Residual risk, stated plainly

- It is a single-maintainer project. Separation of duties is enforced by role checks, not by different people. Accepted for a portfolio.
- There is no edge protection until Phase 5. The local stack is bound to 127.0.0.1 and must not be exposed.
- Deletion of the newest audit records by someone with table-owner rights is undetectable until the chain head is anchored in S3 Object Lock (Phase 4).

- A single HS256 signing key with no `kid` means rotation signs everyone out. Asymmetric keys are a Phase 12 candidate.
- A distributed attacker can still trigger the per-account logout. It is bounded per IP and expires after 15 minutes. Edge bot controls (Phase 5) reduce it further.
- The rate limiter fails closed: if the database is down, sign-in is refused with `503`, not allowed unlimited.

5.3 The control matrix

`docs/security-controls.md` maps each requirement to the threat it counters, the control, the implementation and the evidence a reviewer can run. Each control has an ID by family (C-ID, C-API, C-DB, C-AUD, C-WEB, C-NET, C-CICD, C-GOV, and so on). Three examples:

Requirement	Threat	Control	Implementation	Evidence
Credential stuffing	T-ID-01	WAF rate rule + app limiter + logout + MFA	Auth service; <code>api_policy.LOGIN</code>	<code>test_account_locks_after_repeated_failures</code> , <code>test_login_is_limited_per_ip_with_retry_after</code>
Broken function authorization	T-API-02	Role dependency on every route	<code>core/authz.py</code>	<code>test_authz_matrix.py</code> sweep, including a mutation check
Audit tampering	T-AUD-01	Hash chain + grants + triggers + Object Lock	ADR-0005, migration 0002	<code>test_audit_log.py</code> , <code>make verify-audit</code>

The complete threat register and control catalogue are in Appendices B and C.

5.4 Real, local, simulated, demo

The specification forbids presenting simulated functionality as a real AWS control. Documentation alone drifts, so this is enforced in code (ADR-0009):

Class	Meaning	UI badge
REAL_AWS	Backed by a live, Terraform-managed AWS resource or a real AWS API response	Real AWS (green)
LOCAL	Real functionality running inside SentinelEdge	Local (blue)
SIMULATED	Safe simulation of an attack or control, against SentinelEdge only	Simulated (amber)
DEMO	Synthetic seed data for demonstrations	Demo data (violet)

A capability register (`app/core/capabilities.py`) is served at `GET /api/v1/platform/capabilities` and rendered on every module page. Tests enforce the rules. Nothing can be marked implemented **REAL_AWS** before an AWS phase ships it. Simulator and demo entries can never be **REAL_AWS**. An implemented entry cannot come from a future phase. Wording follows suit: a real control says “Blocked by AWS WAF rule `AWSManagedRulesSQLiRuleSet`”, and a simulated one says “Simulated block — no AWS resource was changed”. After three phases there are, correctly, zero **REAL_AWS** capabilities.

5.5 Governed exceptions

When a control has to be relaxed, it is recorded in a register rather than quietly removed. Each exception names the requester, business justification, risk, compensating controls, approver, exit criteria and expiry. The configuration that implements it carries the exception ID, so the link can be audited in both directions. Two are open, both for developer tooling:

- **EXC-0001:** ESLint is held on major version 9, because the accessibility plugin does not yet support version 10.
- **EXC-0002:** TypeScript is held below version 7, because `typescript-eslint` does not yet support it.

Both expire on 2027-01-07. Phase 10 migrates the register into the application.

Part II — As Built

The completed phases in build order. Each chapter covers what was built and why, how the key mechanisms work, how they are verified, and what was learned.

CHAPTER 6

Phase 1 — Architecture and Secure Foundation

Phase 1 produced no user-facing features. It produced the decisions, the threat model, the repository and a local environment where every later phase inherits enforcement rather than retrofitting it. Release v0.1.0.

6.1 Goals

- Record the target architecture, the threat model and the control matrix before writing features.
- Write the first fourteen ADRs, covering every decision with security trade-offs that later phases depend on.
- Build a local environment whose trust boundaries mirror the AWS design.
- Put cross-cutting security mechanisms (headers, errors, logging, configuration) in place once, for every endpoint that follows.
- Ship real security gates from day one, rather than waiting for the scanning phase.

6.2 Repository layout

```
Sentinel-Edge/  
  backend/           FastAPI application, Alembic migrations, tests  
  frontend/         React SPA, nginx config and security headers  
  db/               bootstrap-roles.sh (used locally, in CI and on RDS)  
  docs/             architecture, threat model, controls, ADRs, runbooks, releases  
  scripts/          smoke test, hardening verification, startup diagnosis  
  terraform/        AWS modules (Phase 3 onward)  
  scenarios/, tools/ attack scenarios (Phase 7), security CLI tools (Phase 11)  
  .github/          CI workflow, Dependabot, CODEOWNERS, PR template  
  docker-compose.yml local topology  Makefile  single entry point
```

6.3 Decisions recorded up front

Phase 1 wrote ADR-0001 to ADR-0014 before any feature code. These include the private origin, single-origin routing, the authentication architecture, two-layer rate limiting, the tamper-evident audit log, Bedrock as the AI provider, the AI output contract, WAF changes through Terraform, provenance classification, CI identity through OIDC, headers at two layers, backend layering, shift-left gates, and Terraform state isolation. Writing them first meant Phases 2 and 6 implemented decisions instead of inventing them under pressure. Each later ADR carries an *as implemented* addendum where reality refined the plan.

6.4 Cross-cutting mechanisms

Security middleware

A pure-ASGI middleware runs on every response, *including* 400, 404, 405 and 500 errors and rejected hosts. It sets the security headers, assigns a correlation ID, removes the server banner, and writes a structured access log containing the path only, never query strings, since those can carry tokens.

Header	API value	Purpose
Content-Security-Policy	<code>default-src 'none'; frame-ancestors 'none'; base-uri 'none'</code>	The API returns JSON only, so nothing may load
Strict-Transport-Security	<code>max-age=63072000; includeSubDomains</code>	Forces HTTPS for two years; <code>preload</code> deferred until the domain is fixed
X-Content-Type-Options	<code>nosniff</code>	No MIME reinterpretation
X-Frame-Options / frame-ancestors	<code>DENY / 'none'</code>	Clickjacking
Referrer-Policy	<code>no-referrer</code>	Security UIs put incident IDs in URLs
Permissions-Policy	<code>camera, microphone, geolocation, payment, USB, sensors off</code>	Limits post-XSS abuse
COOP / CORP	<code>same-origin</code>	Cross-origin isolation
Cache-Control	<code>no-store</code>	Security data never cached

The SPA's nginx applies a strict policy with no `unsafe-inline` and no `unsafe-eval`. Fonts are self-hosted, so no third-party origin appears anywhere in the CSP. `X-XSS-Protection` and `Expect-CT` are deliberately not set: one is deprecated and harmful in old browsers, the other obsolete.

Correlation IDs

An upstream `X-Request-ID` is accepted only if it matches `^[A-Za-z0-9-_.]{8,64}$`. Otherwise a new one is generated. A hostile value therefore cannot be used for log injection or header splitting.

Error envelope

Every error has one shape: `{"error": {"code", "message", "correlation_id", "details?"}}`. Validation errors report the field location and error type, never the submitted value, so input is not reflected. Unhandled exceptions return a generic 500 with a correlation ID. Stack traces, SQL and internal paths go to the log, never to the client.

Structured logging

Logs are JSON to stdout. Keys matching password, secret, token, authorization, cookie, API key, session or private key are replaced with `[REDACTED]`, recursively. Control characters are neutralized to prevent log forging.

Secure-by-default configuration

Settings are validated at startup by Pydantic. Unknown environments are rejected, wildcard Host allow-lists are rejected, DEBUG is refused in deployed environments, and interactive API docs are forced off when deployed. A Host allow-list (`TrustedHostMiddleware`) returns 400 for unexpected Host headers.

Capability register

The provenance enum and capability register (ADR-0009) were built in Phase 1, so every later feature is born with a label.

6.5 Hardened containers

The backend image is a multi-stage build. The builder installs **hash-pinned** dependencies (`pip install --require-hashes --no-deps`) into a virtual environment. The runtime stage copies only that environment and the application, owned by root but run as UID 10001 with no shell. Compose adds a read-only root filesystem, `cap_drop: [ALL]`, `no-new-privileges` and `tmpfs` for `/tmp`. Uvicorn runs with `--no-server-header`, `--no-access-log` (the middleware logs instead) and `--no-proxy-headers`.

`make verify-hardening` proves these properties against the running stack. It checks that every service runs non-root with a read-only root filesystem, no effective capabilities and no-new-privileges; that the database port is not published and its network is internal; that the API and web are bound to localhost; that a forged Host header gets 400; that the API's CSP is deny-all and the SPA's CSP has no `unsafe-inline`; and that no Server header is disclosed.

6.6 Shift-left gates (ADR-0013)

The specification places full scanning in Phase 8. A security project that ran without secret scanning or SAST for seven phases would contradict its own story, so Phase 1 shipped a minimal but real gate set in **pre-commit** and **CI**:

Gate	Tool	Where
Secret scanning (full history)	Gitleaks, detect-private-key	pre-commit, CI
Python lint + security rules	Ruff (including the <code>S</code> bandit family)	pre-commit, CI
SAST	Bandit	pre-commit, CI
Type safety	mypy strict	CI
SCA	pip-audit (hash-pinned), npm audit (<code>--audit-level=high</code>)	CI
Frontend security lint	ESLint bans <code>dangerouslySetInnerHTML</code> , <code>innerHTML</code> , <code>eval</code> , web storage	pre-commit, CI
Coverage floor	pytest <code>--cov-fail-under=90</code>	CI
Branch safety	<code>no-commit-to-branch</code> (no direct commits to main)	pre-commit

The CI workflow runs with `permissions: contents: read`, pins every third-party action to a full commit SHA, and checks out with `persist-credentials: false`. Dependabot proposes updates. CODEOWNERS requires owner review on security-sensitive paths. npm installs use `npm ci --ignore-scripts`, so a dependency cannot run code at install time.

VERIFIED IN PHASE 1

`make check` passes every gate locally. The API runs as a non-zero UID. Writing to the root filesystem fails with "Read-only file system". Port 5432 refuses connections from the host. A forged Host header returns 400. CI is green on the first push.

6.7 Talking points

- "Security gates existed from the first commit. Phase 8 adds scanners, it doesn't introduce scanning."
- "Every error, including 404s and 500s, carries the security headers, and there's a test for each status code."
- "The capability register means the platform can't claim a control it doesn't have, and that's enforced by tests, not by a style guide."

CHAPTER 7

Phase 2 — Secure Application Foundation

Phase 2 built the parts every security platform depends on: who you are, what you may do, and an unalterable record of what you did. It was delivered in eight milestones and released as v0.2.0.

7.1 Milestones

Milestone	Delivered
M0	Dependency hygiene; <code>make verify-hardening</code>
M1	Database foundation with least-privilege roles
M2	Tamper-evident audit log and authentication
M3	RBAC, user management and the audit log API
M4	Frontend: sign-in, setup, user administration, audit viewer
M5	Documentation, end-to-end smoke test, v0.2.0 release notes
M6	Permanent user deletion behind a confirmation dialog (owner request)
M7	Fix for the database-port hardening check on Docker Compose v5

7.2 Database least privilege (ADR-0015)

If the API connected as the schema owner, any SQL injection or code-execution bug would inherit the ability to drop tables, rewrite the audit log, or grant itself anything. Phase 2 therefore uses three identities, each with one job:

Identity	Can	Cannot	Used by
Admin (<code>POSTGRES_USER</code> ; RDS master)	Create roles and the schema	—	<code>db/bootstrap-roles.sh</code> , once
<code>sentinel_migrator</code>	Own and change the <code>sentinel</code> schema	Create roles or databases; superuser; bypass RLS	One-shot migrate container
<code>sentinel_app</code>	Exactly the per-table privileges migrations grant	Any DDL; read <code>alembic_version</code> ; create in <code>public</code>	The API

- **One bootstrap script everywhere.** The same script runs at first start of the local container, in CI, against a throwaway test database, and (Phase 4) against RDS, so the privilege model cannot drift between environments.
- **Explicit grants, no default privileges.** Each migration that creates a table grants the app role only what that table needs. `ALTER DEFAULT PRIVILEGES` is deliberately avoided, because it would hand every future table full access by accident.

- A **privilege-matrix test** compares the app role's actual grants with a hand-written expected table. Over-granting fails CI.
- **Secrets scoped per container.** The API never receives the migrator or admin password. The migrate container never receives the app password.
- **Resource limits** on the app role: `statement_timeout = 15s`, `idle_in_transaction_session_timeout = 30s`.
- **Verified TLS required when deployed** (`sslmode verify-ca` or `verify-full`). Configuration validation refuses to start otherwise.

7.3 Authentication (ADR-0003)

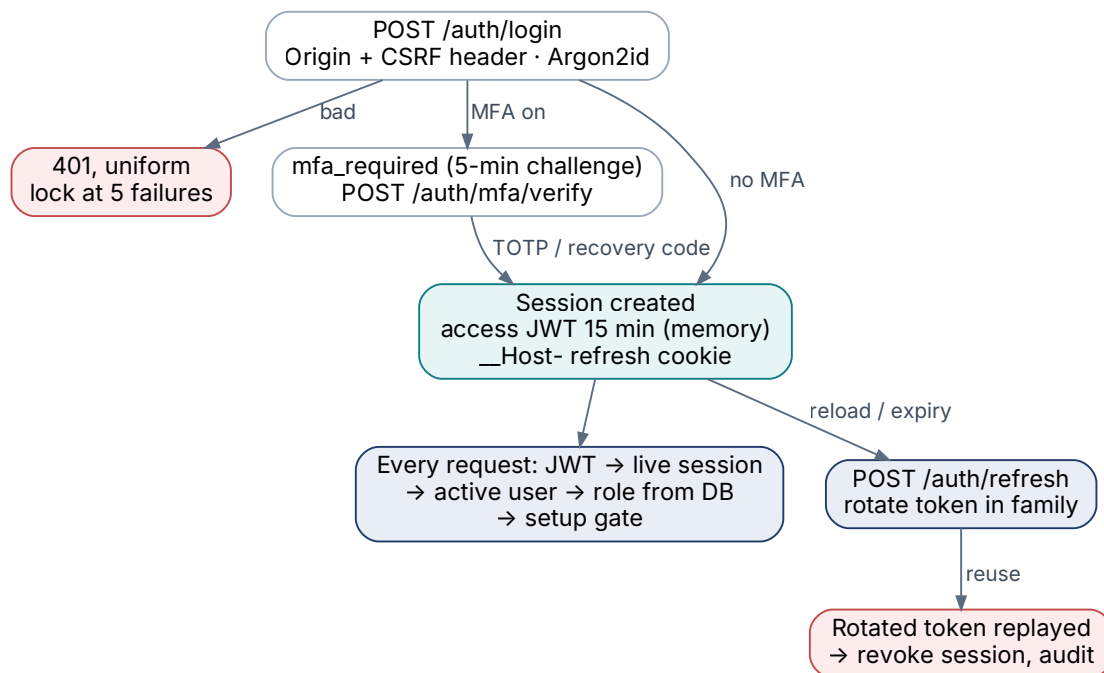


Figure 6. Sign-in, MFA, session and refresh rotation. Red nodes are the defensive outcomes.

Passwords

Passwords are hashed with **Argon2id** (64 MiB, 3 iterations, parallelism 4 by default). Configuration refuses to deploy below the OWASP minimum, and hashes are upgraded on the next sign-in if parameters rise. Unknown accounts are checked against a dummy hash, so a response takes the same work whether the account exists, is locked or is inactive. Every failure returns the same uniform 401.

Tokens and sessions

- The **access token** is a signed JWT with a 15-minute lifetime and the algorithm pinned server-side. `aud`, `iss`, `exp` and token type are all required. The SPA holds it in memory only.
- The **refresh token** is an opaque random value in a `__Host-` cookie: `HttpOnly; Secure; SameSite=Strict; Path=/`, with no `Domain`. The `__Host-` prefix guarantees no subdomain can set or overwrite it. The server stores only its hash.

- Refresh tokens **rotate on every use** within a family. Presenting an already-rotated token means it was stolen and replayed, so the whole session is revoked and the event is audited as `auth.refresh_token_reuse`.
- **Revocation is immediate.** Every access token carries its session ID, and each request loads that session, so logout, password change, role change and deactivation take effect at once rather than when the token expires. The role is read from the database on every request.

Multi-factor authentication

TOTP secrets are Fernet-encrypted with a key held outside the database, so a database dump does not yield working second factors. The last accepted time-step is stored, so a code works exactly once (no re-play within its 30-second window). Recovery codes are hashed and single-use. MFA is mandatory for ADMIN and SECURITY_ENGINEER. The setup QR code is rendered locally in the browser, so the secret never goes to a third-party QR service.

Lockout, CSRF and recovery

- Five failures lock an account for 15 minutes. Password and MFA failures share the counter, and attempts are counted under a `SELECT ... FOR UPDATE`, so concurrent guesses are not lost.
- Cookie-bearing endpoints (sign-in, MFA verify, refresh, password reset) require an allowed `Origin` and a custom `X-SentinelEdge-CSRF` header. Browsers will not attach a custom header cross-site without a CORS preflight, and the API grants none.
- Password reset uses a single-use, hashed, 30-minute token carried in the URL **fragment**, which is never sent to servers or logged. The SPA strips it from the address bar, and a successful reset revokes all sessions.

No default credentials

The first admin is created by `make create-admin`, which prints a one-time password, shown once. Everyone else is **invited** by a one-time link (72-hour expiry) delivered through a local outbox. Admins never see or set anyone's password. A new or reset account is held at a setup gate: until the password is changed (and, for privileged roles, MFA enrolled), only `/auth/me`, `/auth/logout`, `/auth/password/change` and the enrollment endpoints respond.

7.4 Authorization

Five roles: ADMIN, SECURITY_ENGINEER, DEVELOPER, ANALYST and VIEWER. Every route depends on exactly one of three guards: `public_endpoint`, `authenticated_setup` or `require_roles(...)`. A route with none fails the test suite.

`tests/security/test_authz_matrix.py` holds the full endpoint-by-role matrix (Appendix D) as an independent, hand-written table. It calls every protected route anonymously (expecting 401) and as every role (expecting 403 exactly where the table says no). A deliberate mutation, weakening one route, fails three tests. That proves the sweep catches drift.

Object-level authorization (OWASP API1) lives in the service layer. Non-admins may read only their own user record. Any other ID returns the *same* 404 as a non-existent one, so existence is not leaked, and the attempt is audited as `authz.denied`. Admins cannot demote, deactivate, delete or reset MFA on themselves, and the last active admin cannot be removed.

Deactivate versus delete

Deactivation is reversible, revokes sessions at once, and keeps the account for investigation; it is the preferred response during an incident. Deletion (the trash icon, behind a warning dialog whose default action is Cancel) removes the account, sessions and pending links permanently. **Audit history is unaffected**, because audit records hold the actor's ID and email as plain values rather than foreign keys, so the hash chain never changes.

7.5 Tamper-evident audit log (ADR-0005)

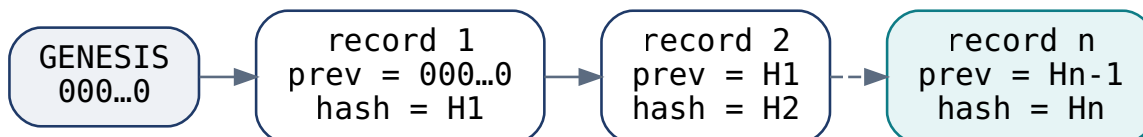


Figure 7. The audit hash chain. Altering any record breaks every hash after it. Each hash covers the record and the previous hash.

Every record carries the actor, timestamp, action, resource, result, source IP and correlation ID. Its hash covers the previous record's hash:

```
def compute_record_hash(prev_hash: str, payload: dict[str, Any]) -> str:
    canonical = json.dumps(payload, sort_keys=True, separators=(",", ":"), ensure_ascii=False)
    return hashlib.sha256(f"{prev_hash}\n{canonical}".encode()).hexdigest()
```

Four layers protect it:

1. **Hash chain.** Verification walks the chain and reports the first broken record. It is available as `make verify-audit`, `GET /api/v1/audit-logs/verify` (itself audited) and a “Verify integrity” button in the UI.
2. **Grants.** The app role has only `SELECT` and `INSERT` on `audit_log`.
3. **Triggers.** `UPDATE`, `DELETE` and `TRUNCATE` raise an error for *every* role, including the table owner. Getting around that requires `ALTER TABLE ... DISABLE TRIGGER`, which is DDL.
4. **Off-host anchor** (Phase 4). The chain head is exported to S3 Object Lock, which closes the one gap a chain cannot close: silent deletion of the newest records.

Writers take `pg_advisory_xact_lock` before reading the chain head, so concurrent requests extend the chain one at a time (tested with eight concurrent writers). Details are redacted and size-bounded *before* hashing, so a secret never enters the chain.

7.6 Frontend

The Phase 2 UI provides sign-in with MFA, a forced setup flow with local QR rendering, password reset, user administration (invite, change role, deactivate, reset 2FA, delete), and an audit-log viewer with filters and integrity verification. The UI hides what a role cannot use, for usability. That is never the control.

One subtle problem had to be solved. Two open tabs refreshing at the same moment would present the same refresh token twice and trip reuse detection, logging the user out (T-ID-06). Refresh is therefore single-flight within a tab and serialized across tabs with the **Web Locks API**.

TWO REAL BUGS, FOUND BY THE TESTS

Unicode digits. The MFA check used `\d`, which in Python matches non-ASCII digits such as Arabic-Indic numerals. Those passed validation and then crashed the constant-time comparison with a 500. The fix is ASCII-only `[0-9]`, with a regression test (T-INP-01). **Role identity.** Roles loaded from the database were plain strings, so an `is` comparison against the enum never matched. That kind of check can silently fail open or closed. The fix is an enum-typed column and `==` comparisons, caught by the authorization sweep (T-AZ-01).

VERIFIED IN PHASE 2

302 backend tests (99% coverage) against real PostgreSQL, and 41 frontend tests. `make smoke`: 26 end-to-end checks against the running stack. In headless Chromium, the full first-run journey left web storage empty, the refresh cookie was invisible to JavaScript, and there were no console errors.

CHAPTER 8

Phase 6 — API Security

Phase 2 decided who may call each endpoint. Phase 6 adds how often, makes the whole API surface visible, and proves coverage of the OWASP API Security Top 10 with tests that can be run. Released as v0.3.0.

8.1 Milestones

Milestone	Delivered
M0	Housekeeping before API security work
M1	PostgreSQL token-bucket rate limiting and trusted client IPs
M2	API inventory, endpoint metrics and OWASP API Top 10 coverage
M3	API Security Center page (/apis)
M4	Documentation, smoke test (34 checks), v0.3.0 release notes
M5	Reliable smoke burst, 503 when the limiter's database is down, settled test renders

8.2 One registry for every endpoint

`backend/app/core/api_policy.py` gives every route a **risk rating**, a **rate-limit policy**, its **OWASP API Top 10 exposure**, the **data it handles** and any **object-level rule**. The limiter and the inventory both read this registry, so what the inventory reports is what the code enforces. A test fails if the registry and the live route table differ in either direction (OWASP API9: improper inventory management). Authentication and authorization are *not* duplicated in the registry. They are read from each route's own guard dependencies, so they cannot disagree with them.

8.3 Rate limiting (ADR-0017)

The policies

Policy	Limit	Keyed by	Endpoints
<code>ip_global</code>	600 / min	IP	every API route, before authentication
<code>probe</code>	120 / min	IP	<code>/health</code> , <code>/ready</code>
<code>login</code>	20 / 2 min	IP	<code>POST /auth/login</code>
<code>mfa_verify</code>	20 / 2 min	IP	<code>POST /auth/mfa/verify</code>
<code>token_refresh</code>	30 / min	IP	<code>POST /auth/refresh</code>
<code>account_recovery</code>	5 / 15 min	IP	password forgot and reset
<code>session</code>	60 / min	user	<code>/auth/me</code> , <code>/auth/logout</code>

Policy	Limit	Keyed by	Endpoints
credential_change	10 / 5 min	user	password change, MFA enrollment
read	120 / min	user	list and read endpoints
admin_write	30 / min	user	user administration
expensive	6 / min	user	audit chain verification

An atomic token bucket in one SQL statement

Each bucket is one row per (policy, subject). A check is a single `INSERT ... ON CONFLICT DO UPDATE` that refills the bucket for elapsed time, spends a token if one is available, and reports the outcome, all on **the data-base's clock**:

```
INSERT INTO sentinel.rate_limit_buckets AS b
(bucket_key, tokens, updated_at, last_allowed, denied_since)
VALUES (:key, :capacity - 1, now(), true, NULL)
ON CONFLICT (bucket_key) DO UPDATE SET
  tokens = CASE
    WHEN LEAST(:capacity, b.tokens + EXTRACT(EPOCH FROM now() - b.updated_at) * :rate) >= 1
    THEN LEAST(:capacity, b.tokens + EXTRACT(EPOCH FROM now() - b.updated_at) * :rate) - 1
    ELSE LEAST(:capacity, b.tokens + EXTRACT(EPOCH FROM now() - b.updated_at) * :rate)
  END,
  last_allowed =
    LEAST(:capacity, b.tokens + EXTRACT(EPOCH FROM now() - b.updated_at) * :rate) >= 1,
  denied_since = CASE ... ELSE COALESCE(b.denied_since, now()) END,
  updated_at = now()
RETURNING tokens, last_allowed, (denied_since = updated_at) AS first_denial
```

Because the row lock taken by the upsert serializes concurrent checks, a bucket can never be overspent, whichever API instance handles the request. A 40-thread concurrency test proves it. The check runs on its own connection and commits immediately, so a request that later fails has still spent its token. Using the database clock means API instances with skewed clocks still agree.

WHY A TOKEN BUCKET, NOT A FIXED WINDOW?

Fixed-window counters allow a double burst at window edges: a full window's worth just before the boundary, then another just after. A token bucket refills continuously, so the burst is bounded by the bucket's capacity at all times.

Two checkpoints

- **Per-IP, before authentication.** A router-level dependency on every API route applies the global ceiling plus the endpoint's own IP-keyed policy. A sign-in flood is rejected before any Argon2id hashing is spent on it.
- **Per-account, after authentication.** Applied right after the session is authenticated, so the limit follows the user across IP addresses.

Responses and auditing

A denied request gets `429` with `Retry-After` and the standard error envelope. Allowed requests carry `Rate-Limit-Limit`, `RateLimit-Remaining` and `RateLimit-Reset`, following the IETF draft. Only the **first denial in a run** is audited (`ratelimit.exceeded`, with policy, scope and endpoint). Later denials are counted in metrics, so a flood cannot also flood the audit log (T-API-13).

Guard rails

Rate limiting cannot be disabled in deployed environments; configuration validation refuses. Idle buckets are pruned after a day. If the limiter cannot reach its database it **fails closed**: the request is refused with `503` and `Retry-After: 5`, never allowed through unlimited.

8.4 Trusting the right client IP

Every browser request reaches the API through a proxy (nginx locally; CloudFront and the ALB on AWS). Without proxy-header trust, every client appears to come from the proxy, so per-IP limits throttle everyone together and audit records name the proxy. Trusting `X-Forwarded-For` from anyone is worse: a client could choose its own address (T-ORG-02).

`ClientIpMiddleware` runs outermost and rewrites the ASGI client address once:

1. If the direct TCP peer is **not** inside `SENTINEL_TRUSTED_PROXY_CIDRS`, the header is ignored and the peer is the client.
2. Otherwise the header is walked **right to left**, skipping trusted proxies. The first untrusted address is the client.
3. A malformed hop stops the walk. Nothing to its left can be trusted.

Anything a client prepends to the header sits on the *left*, so it is never reached. Configuration rejects trust-all networks. Locally the edge Docker network is pinned to `172.30.86.0/24`, nginx overwrites the header with `$remote_addr`, and Uvicorn's own proxy handling is switched off, so there is exactly one decision point. Every consumer (rate limiting, audit, access logs) reads the same resolved address.

PROVEN END TO END

Twenty-two sign-in attempts were sent through nginx with a spoofed `X-Forwarded-For: 6.6.6.6`. The limiter returned 401s and then 429s with `Retry-After`. Exactly one `ratelimit.exceeded` audit record was written, naming the real client (172.30.86.1), not 6.6.6.6.

8.5 Route templates

FastAPI keeps included routers nested, so at request time the matched route knows only its router-relative path (`/ {user_id}`), not the endpoint (`/api/v1/users/{user_id}`). At startup the app builds a map from each route object to its full template. The limiter, metrics and audit records all use it. This also fixed a Phase 2 defect: authorization-denial records had named router-relative paths and now record the full endpoint.

8.6 Endpoint metrics

`ApiMetricsMiddleware` keeps hourly counters per route template in `api_endpoint_stats`: requests, client and server errors, 401s, 403s and 429s. Unmatched paths are counted together as "(unmatched)", a sign of someone probing for undocumented endpoints, and only the count is kept, never the path. **No IP addresses, user IDs, path parameters, query strings or payloads are stored**, so the table holds no personal data. Rows older than 30 days are pruned. Writes run in a worker thread, and a failure to write metrics never breaks a request.

8.7 The API Security Center

The `/apis` page (ADMIN, SECURITY_ENGINEER and DEVELOPER) is backed by `GET /api/v1/api-security/inventory` and `GET /api/v1/api-security/owasp`. For every endpoint it shows the method and path, authentication, authorization (including CSRF and object rules), risk, rate limit, 24-hour requests, error rate, security rejections (401 + 403 + 429), last scan and a computed status:

- **protected** — controls in place, nothing notable;
- **elevated** — requests were throttled, or there were five or more forbidden responses in 24 hours;
- **review** — there were server errors.

The inventory's access description for every endpoint is cross-checked in a test against the independently written authorization matrix from Phase 2. Two separate sources must agree.

8.8 OWASP API Security Top 10 (2023)

Category	Status	How
API1 Broken Object Level Authorization	Mitigated	Service-layer ownership checks; others' records return 404; denials audited
API2 Broken Authentication	Mitigated	Argon2id, MFA, short-lived tokens checked per request, refresh reuse detection, logout, per-IP sign-in limits
API3 Broken Object Property Level Authorization	Mitigated	Response models allow-list fields; request models forbid unknown fields; route-table sweeps enforce both
API4 Unrestricted Resource Consumption	Partial	Token buckets, page-size caps, 1 MB body limit, 15 s statement timeout. WAF rate rules in Phase 5
API5 Broken Function Level Authorization	Mitigated	Roles declared on every route; sweep as every role
API6 Unrestricted Access to Sensitive Business Flows	Partial	Sign-in, MFA, recovery and invitations limited; no enumeration. WAF Bot Control in Phase 5
API7 Server Side Request Forgery	Not exposed	No endpoint fetches user URLs; egress guard ready for Phase 9
API8 Security Misconfiguration	Partial	Headers everywhere, Host allow-list, generic errors, docs off when deployed, hardened containers. Edge TLS in Phase 5
API9 Improper Inventory Management	Mitigated	Inventory generated from the route table; registry must match; probes counted
API10 Unsafe Consumption of APIs	Not exposed	No third-party APIs consumed; Bedrock output contract in Phase 9

Each category in the live endpoint cites the specific tests that prove it, and a test fails if any cited test stops existing. “Partial” categories state what completes them. The claims cannot outrun the evidence.

8.9 Route-table sweeps

Three tests walk the whole route table rather than sampling it:

- every request body model forbids unknown fields (mass assignment);
- every JSON route declares a response model (excessive data exposure);
- any response field whose name *looks* sensitive (hash, secret, token, password...) needs explicit, reasoned approval. Four were approved: `prev_hash`, `record_hash` and `head_hash` (audit verification needs them) and `must_change_password` (a boolean flag).

8.10 SSRF guard

`app/security/egress.py` must approve any future outbound request. It requires HTTPS on the default port with no credentials in the URL, prefers an explicit host allow-list, and refuses a destination if **any** resolved address is not public. That covers loopback, RFC 1918, link-local (including the metadata service at 169.254.169.254), carrier-grade NAT, multicast, reserved and unique-local IPv6, and IPv4-mapped or 6to4 forms of all of them. Callers connect to the approved addresses rather than doing a fresh DNS lookup, which defeats DNS rebinding. Nothing calls out yet. The guard is ready, and tested, before the first integration needs it.

FIXED DURING THE PHASE

The smoke test’s probe flood sent a fixed 125 requests, and on a slower machine the bucket refilled mid-burst and no 429 came back. It now sends until the first 429, which arrived after about 130 requests on the owner’s machine. The limiter also originally answered a database outage with a generic 500. It now returns 503 with `Retry-After`, still refusing the request.

VERIFIED IN PHASE 6

391 backend tests (99% coverage) and 45 frontend tests. `make smoke`: 34 checks, including a real rate-limit trip, an ignored spoofed `X-Forwarded-For`, and the inventory counting the throttled requests. `make verify-hardening`: 17 checks. Ruff, mypy (strict), Bandit, ESLint, TypeScript and the schema-drift check are clean. Verified in headless Chromium against a live API, PostgreSQL and nginx, with nothing in web storage.

Part III — The Road Ahead

The remaining phases as designed. Decisions already recorded in ADRs are noted. Each chapter is rewritten in full when its phase ships and moves into Part II.

CHAPTER 9

Phase 7 — Security Operations

Phase 7 turns the controls built so far into something an analyst can watch and act on: security events, a dashboard, incidents worked from detection to closure, and a safe attack simulator. **NEXT**

9.1 Scope

- **Event collection.** A shared `security_events` store with a provenance label on every record. The first sources already exist: sign-in failures, lockouts, refresh-token reuse, authorization denials, rate-limit trips and unknown-path probes from Phases 2 and 6. WAF log ingestion arrives as a `REAL_AWS` source in the AWS window.
- **Security dashboard.** Overall posture, active threats, blocked and suspicious requests, open incidents and API risk. Traffic by method, response code, endpoint and source. Threats by class: SQL injection, XSS, SSRF, path traversal, command injection, credential stuffing, bot activity, API abuse and rate-limit violations.
- **Incident management.** The workflow DETECTED → TRIAGED → INVESTIGATING → CONTAINMENT → REMEDIATION → VALIDATION → CLOSED. Each incident has an owner, a timeline, related events, evidence, analyst notes and remediation, and every transition is audited.
- **Attack simulator.** Controlled, clearly labelled SIMULATED events (SQL injection, XSS, path traversal, SSRF, credential stuffing, bot activity, API abuse, suspicious authentication, expired certificate, vulnerable dependency). They target only SentinelEdge, never an external system.
- **Application inventory.** Protected applications with owner, environment, criticality, domain, API count, WAF and certificate status, security score, last scan and vulnerability count.
- **Background worker.** The same image as the API with a separate endpoint, for ingestion and scheduled jobs.

9.2 Design commitments already made

CARRIED IN FROM EARLIER PHASES

- The incident record must preserve evidence integrity. Transitions are written to the hash-chained audit log (ADR-0005).
- Simulated events can never be labelled `REAL_AWS` (ADR-0009). The UI says “Simulated”, and the capability register’s tests enforce it.
- In-dashboard WAF rule toggles exist only inside the simulator. Real changes go through Terraform pull requests (ADR-0008).
- Analyst-entered text is rendered as text, never as HTML, consistent with the Phase 1 lint bans.

9.3 Security focus

Detection logic gets its own tests (each rule fires on its attack and stays quiet on benign traffic). Incident state transitions are enforced server-side by role. ANALYST can investigate and add notes, and closure requires SECURITY_ENGINEER or ADMIN. The simulator is labelled at every layer: API, database and UI.

CHAPTER 10

Phase 8 — Application Security Scanning

Phase 1 shipped baseline gates. Phase 8 completes the scanner set, generates an SBOM on every build, and turns findings into a managed vulnerability lifecycle with SLAs. **PLANNED**

10.1 Scope

Category	Tool	Adds to Phase 1 baseline
SAST	Semgrep (+ Bandit, Ruff S)	Cross-language rules, custom rules for project patterns
SCA	pip-audit, npm audit	Results ingested as findings, not just gates
Secrets	Gitleaks	Unchanged; findings recorded
Container	Trivy	Image OS and library CVEs; misconfiguration checks
IaC	Checkov	Every Terraform module scanned before Phase 3 applies anything
DAST	OWASP ZAP	Authenticated scans against the local stack; fills "Last scan" in the API Security Center
SBOM	Syft	CycloneDX per build: dependencies, versions, licenses, known vulnerabilities

10.2 Vulnerability management

Each finding records an ID, source, severity, CVSS, affected component, detection date, status, owner, remediation recommendation, due date and evidence. Statuses are Open, Triaged, In Progress, Remediated, Accepted Risk and False Positive. Severity drives a due date. **Critical and high findings fail the pipeline by default.** The only way past the gate is a risk-acceptance record with requester, justification, compensating control, approver and expiry, which Phase 10 models in the application.

DESIGN NOTES

- Scanners must be configured and producing results, not merely listed (spec §19).
- SBOM status appears on the dashboard. A vulnerable dependency becomes both a finding and, through the simulator path, a demonstrable event.

CHAPTER 11

Phase 10 — Threat Modeling and Governance

The threat model and control matrix have so far lived in Markdown. Phase 10 brings them into the application, alongside exceptions, risk acceptance and change management.

PLANNED

11.1 Scope

- **Threat modeling module.** STRIDE and PASTA, with assets, trust boundaries, data flows, threats, attack paths, controls and residual risk. The first model in the module is SentinelEdge's own, migrated from `docs/threat-model.md` rather than retyped.
- **Control catalogue and mappings.** Controls linked to threats, requirements and evidence, so the requirement → threat → control → implementation → evidence matrix is queryable.
- **Exceptions and risk acceptance.** Every exception names a requester, business justification, risk, compensating control, approver and expiry. Expiry is enforced, never left to lapse. EXC-0001 and EXC-0002 migrate in from `docs/governance/exceptions.md`.
- **Change management.** Security-sensitive changes produce a change ID, requester, description, risk, approval, implementation, validation and rollback plan. WAF change requests (ADR-0008) are the first consumer.
- **Security posture score.** Scored by category (network, WAF, API, identity, vulnerability management, TLS, logging, monitoring, AI, DevSecOps, governance). The score must be **explainable**: each number links to the controls and evidence behind it, never an arbitrary or AI-generated figure.

SEPARATION OF DUTIES

A requester cannot approve their own exception or change. On a single-maintainer project this is enforced by role checks and recorded in the audit log, and that limitation is stated openly.

CHAPTER 12

Phase 9 — AI Security Engine

AI in SentinelEdge is a security-intelligence component, not a chatbot. It analyzes events and findings, explains risk and recommends controls, and it is never allowed to act on its own.

PLANNED

12.1 Scope

The engine analyzes WAF events, API events, HTTP requests, security logs, vulnerability findings, threat-model findings and incident data. It produces summaries, risk explanations, attack classifications, remediation recommendations and threat-model suggestions, using Amazon Bedrock through the IAM task role (ADR-0006).

12.2 The output contract (ADR-0007)

Attacker-controlled content (request bodies, headers, log lines) flows into the analysis, so indirect prompt injection is the primary threat (OWASP LLM01). The defenses:

- **Untrusted data is delimited, never concatenated into instructions**, after input validation and prompt-risk scoring.
- **Structured output.** The model must return JSON matching a Pydantic schema. `observed_evidence` (verbatim facts from the event) is kept separate from `inference` (the model's interpretation), alongside classification, severity and confidence. Non-conforming output is **rejected, not repaired**, and the UI says so.
- **Output handling.** AI text is rendered as text only, never as HTML or Markdown-to-HTML, and never executed or used to build queries.
- **No agency.** The AI has no tools that change state. It can only create an `ai_action_proposal`. High-impact actions need SECURITY_ENGINEER or ADMIN approval, which is audited.
- **Data minimization and cost.** Only redacted, minimized event data is sent, with no credentials, tokens or PII fields. Every invocation is audit-logged with model, token counts and prompt-risk score, and bounded by per-user quotas (LLM10).
- **Safe egress.** Outbound calls pass the Phase 6 SSRF guard.

TEST PLAN

A prompt-injection corpus (direct, indirect via log lines, encoded and multi-step) must fail to change classifications or produce proposals outside the schema. The `offline` provider makes these tests deterministic and free. Bedrock is the one AWS service used before the AWS window: on-demand calls, pennies, no infrastructure.

CHAPTER 13

Phases 3–5 — AWS Foundation, Deployment and Edge

The three AWS phases run as one focused deployment window after the local work is finished. The Terraform is written and statically scanned before anything is applied. **PLANNED**

13.1 Phase 3 — Terraform AWS foundation

VPC with public, private-application and isolated-data subnets; routing; tiered security groups; IAM roles per function; ECR; and the Terraform state architecture (ADR-0014):

- **State.** An S3 backend with native lockfiles (`use_lockfile = true`, Terraform $\geq$ 1.10). The bucket is SSE-KMS encrypted and versioned, with public access blocked, a TLS-only policy and access logging. A small `bootstrap/` stack creates it.
- **Per-environment roots.** `backend.tf` and `providers.tf` live inside each `environments/<env>/` directory, not at the root. A shared root backend is the most common way development work lands on production state.
- **Guards.** `allowed_account_ids` on every provider, `default_tags` stamping the environment, and CI roles scoped by OIDC `sub`.
- **Checkov clean** on every module: no public databases, no open security groups, no unencrypted storage, no missing logging.

13.2 Phase 4 — Application deployment

ECS Fargate for the API and worker, an **internal** ALB (HTTPS, desync mitigation “strictest”, invalid headers dropped), and RDS PostgreSQL in isolated subnets with KMS and `rds.force_ssl`. Secrets Manager replaces `.env`, scoped per task as it is per container locally. CloudWatch logs and alarms. Two items close Phase 2 caveats:

- The RDS master is granted the membership needed to transfer schema ownership, and the **same** `bootstrap-roles.sh` runs as a one-off task.
- The audit chain head is exported to **S3 with Object Lock**, anchoring the log outside the database.

Client-IP trust switches from the local edge subnet to the ALB subnet ranges, with no code change, only configuration.

13.3 Phase 5 — CloudFront, WAF and TLS

Route 53 (with CAA), CloudFront with ACM certificates (TLS 1.2+), the SPA from private S3 through Origin Access Control, and `/api/*` through a **VPC origin** to the internal ALB. AWS WAF gets managed rule groups, custom rules and rate-based rules scoped per path (`/api/v1/auth/*` tight, `/api/*` general). A CloudFront response-headers policy carries the SPA headers, with a test comparing it to the local nginx policy. Validation proves the origin is **unreachable from the internet**, and WAF rules are tested against the project's own domain only. Certificate monitoring with expiry alerts feeds the dashboard.

COST DISCIPLINE

An always-on development environment is roughly \$110–125 per month. The plan is deploy, capture evidence (plan/apply output, WAF blocks, origin-unreachable proof, a recorded walkthrough) into `docs/evidence/`, then destroy. Re-creating it for an interview costs a few dollars.

CHAPTER 14

Phases 11–12 — Automation and Final Hardening

The last two phases join everything into one gated pipeline and then review the whole system as an assessor would. `PLANNED`

14.1 Phase 11 — Automation and DevSecOps

The full pipeline: commit → unit tests → lint → SAST → SCA → secret scan → IaC scan → SBOM → container build → container scan → push to ECR → Terraform validate → Terraform plan → deployment → DAST → post-deployment security validation. Thresholds for critical and high findings are configurable, and the pipeline fails on them.

CI reaches AWS through **GitHub OIDC** (ADR-0010). There are no long-lived keys. Trust policies pin `aud` and a `sub` scoped to a GitHub *environment*, and production requires reviewer approval. Python automation tools ship with CLI examples: `security_report.py`, `certificate_inventory.py`, `api_inventory.py`, vulnerability reporting, WAF event analysis and posture calculation.

14.2 Phase 12 — Portfolio hardening

A full security review across dependencies, IAM, Terraform, WAF, API, AI and containers. Final documents are produced: `SECURITY_ASSESSMENT.md`, `ARCHITECTURE.md`, `THREAT_MODEL.md`, `INTERVIEW_DEMO.md` and `LESSONS_LEARNED.md`. Known hardening candidates already recorded include asymmetric JWT keys with rotation (`kid`), the database load of rate-limit and metrics writes, and the remaining runbooks.

14.3 Demo mode

The primary interview demonstration runs end to end. A suspicious request is simulated, the WAF detects and blocks it, a security event appears, an analyst opens it, AI analyzes it, an incident is created and investigated, a remediation is recommended and documented, and the dashboard updates. Five documented scenarios (SQL injection, credential stuffing, API abuse, prompt injection, certificate expiration) each cover attack, detection, control, evidence, investigation, remediation and validation.

14.4 Acceptance criteria

The specification declares SentinelEdge complete only when all of these hold:

Criterion	Status	Criterion	Status
Runs locally	✓	Authentication works	✓
RBAC works	✓	API security controls work	✓
Audit logging works	✓	Secret scanning works	✓ (baseline)

Criterion	Status	Criterion	Status
SAST / SCA work	✓ (baseline)	Security runbooks exist	started
Security events are logged	P7	Incident response workflow	P7
Attack simulations work	P7	DAST, IaC, container scanning, SBOM	P8
Vulnerability management	P8	Threat modeling exists	✓ docs; app P10
AI functionality, prompt-injection defenses	P9	Deploys to AWS, Terraform-managed	P3–4
Private database and containers	P3–4	Secrets securely managed	P4
CloudFront, WAF, HTTPS/TLS	P5	Certificate monitoring	P5
IAM least privilege	P3, P11	CI/CD pipeline and security gates	✓ baseline; P11
Terraform modules documented	P3	Demo mode works	P12

Part IV — Reproducing the Build

The commands, in order, that bring SentinelEdge up from an empty machine, verify every control, and ship a release the same way each phase has shipped.

CHAPTER 15

Reproducing the Build

These are the commands, in order, that bring SentinelEdge up from an empty machine and prove its controls. Each step says why it comes where it does. They were run on Garuda Linux with the fish shell. Every project command is a `make` target and behaves the same in bash or zsh. The few fish-specific lines are marked.

15.1 Prerequisites

Tool	Version	Used for
Docker + Compose v2 (v5 tested) + buildx	current	The local stack
Python (via <code>uv</code>)	3.12	Backend tooling and tests, matching CI
Node.js + npm	22 LTS or newer	Frontend tooling and tests
GNU Make, OpenSSL, git, unzip	any	Entry points, secret generation
pre-commit	current	Local security gates
GitHub CLI (<code>gh</code>)	current	Pull requests and releases
Terraform $\geq$ 1.10, AWS CLI v2	—	Not needed until Phase 3

15.2 Step 1 — Prepare the machine

Install the toolchain and let your user run Docker without `sudo`. This keeps every `make` target sudo-free, and files the tools create stay owned by you. Group membership only applies to a new login session.

```
sudo pacman -S --needed git unzip make openssl docker docker-compose docker-buildx \
    nodejs npm uv pre-commit github-cli          # Arch/Garuda; use your distro's equivalents
sudo systemctl enable --now docker.service
sudo usermod -aG docker $USER
# log out completely and back in, then:
docker run --rm hello-world
```

15.3 Step 2 — Get the source

```
mkdir -p ~/Sentinel-Edge && cd ~/Sentinel-Edge
git clone https://github.com/Bionic-Hacker/Sentinel-Edge.git sentineledge
cd sentineledge
git log --oneline -3          # main ends at the latest release merge
```

15.4 Step 3 — Create the Python environment and install dependencies

CI runs linters, type checkers and scanners against the source, so local results only match CI if the versions match. `make install` installs hash-pinned Python packages (`--require-hashes`) and the exact npm lockfile, without running install scripts.

```
uv venv --python 3.12 --seed .venv
source .venv/bin/activate.fish      # fish; in bash/zsh: source .venv/bin/activate
make install
make precommit                     # git hooks: Gitleaks, Ruff, Bandit, ESLint
```

15.5 Step 4 — Generate local secrets

There are no default credentials anywhere in the project. `make env` copies `.env.example`, replaces every placeholder with a distinct random value, and sets mode 600. It never overwrites an existing `.env`. Compose refuses to start without it, so it must come before the stack.

```
make env
ls -l .env                          # -rw-----
```

15.6 Step 5 — Start the stack

`make dev` builds the images and starts them in dependency order. `db` initializes and runs the role bootstrap, `migrate` applies all migrations as `sentinel_migrator` and exits, then `api` (as `sentinel_app`) and `web` start. If startup fails, `make dev` prints a diagnosis with the fix, for example a stale database volume from an older phase.

```
make dev
docker compose ps -a                # migrate Exited (0); db, api, web running
curl -s http://localhost:8000/api/v1/ready
xdg-open http://localhost:8080
```

15.7 Step 6 — Create the first administrator

`create-admin` prints a one-time password, shown once. Signing in with it forces a new password and, for the admin role, authenticator-app enrollment. Save the recovery codes it shows.

```
make create-admin EMAIL=you@example.com
```

Then, in the browser, open **Settings** → **Users** and invite a second user, for example an ANALYST. Invitations land in the local outbox rather than a real mailbox:

```
make outbox                        # shows the one-time invitation link
```

15.8 Step 7 — Prove the controls

Each check below demonstrates a control from the threat model on your own machine. These are the checks an interviewer may ask you to show.

```
make check          # every CI gate: lint, types, tests, SAST, SCA, secret scan
make smoke          # 34 end-to-end checks against the running stack
make verify-hardening # 17 container and network checks
make verify-audit    # walk the audit hash chain; prints the head hash
```

Watch per-IP rate limiting, and spoofing being ignored, through the edge (fish syntax):

```
for i in (seq 22)
  curl -s -o /dev/null -w "%{http_code} " -X POST http://localhost:8080/api/v1/auth/login \
    -H 'Content-Type: application/json' -H 'Origin: http://localhost:8080' \
    -H 'X-SentinelEdge-CSRF: 1' -H 'X-Forwarded-For: 6.6.6.6' \
    -d '{"email":"nobody@example.com","password":"wrong"}'
end; echo
```

Expected: up to twenty 401 responses, then 429. Under **Audit Logs**, filter by `ratelimit.exceeded`. There is one record for the whole burst, naming your real address (172.30.86.x), not 6.6.6.6. Then open **/apis**: the login row shows *Elevated* with the throttled count. Signed in as the analyst, the same page explains that the inventory is restricted, and no request for it is made.

Container hardening by hand:

```
docker compose exec api id -u          # non-zero
docker compose exec api touch /probe   # Read-only file system
nc -zv 127.0.0.1 5432                  # Connection refused
curl -s -o /dev/null -w "%{http_code}\n" -H 'Host: evil.example' http://localhost:8000/api/v1/health
# 400
```

15.9 Step 8 — Ship a phase the way this project does

This is the release workflow from Chapter 2. The commands are the ones used for v0.1.0 through v0.3.0, shown here for the next release (Phase 7, v0.4.0). The GitHub token is a fine-grained token scoped to this one repository (Contents, Workflows and Pull requests read/write; Actions read-only). It is pasted at the prompt and never written to disk.

```
git switch -c phase/7-security-ops      # one branch per phase
# ... milestone commits; make check && make smoke && make verify-hardening ...
git push -u origin phase/7-security-ops  # password prompt: paste the token

read -s -g -x -P "GitHub token: " GH_TOKEN # fish: hidden prompt, exported for gh
gh pr create --repo Bionic-Hacker/Sentinel-Edge --base main --head phase/7-security-ops \
  --title "Phase 7: security operations" --body-file docs/releases/v0.4.0.md
gh pr checks phase/7-security-ops --repo Bionic-Hacker/Sentinel-Edge --watch
gh pr merge phase/7-security-ops --repo Bionic-Hacker/Sentinel-Edge --merge

git switch main && git pull --ff-only
git tag -a v0.4.0 -m "Phase 7: security operations" # SSH-signed (tag.gpgsign true)
git push origin v0.4.0
gh release create v0.4.0 --repo Bionic-Hacker/Sentinel-Edge \
  --title "v0.4.0 - Phase 7: security operations" --notes-file docs/releases/v0.4.0.md
set -e GH_TOKEN # fish: forget the token
```

Signed tags use an SSH signing key. Set it up once, and register the public key on GitHub as a **Signing Key** so tags show as Verified:

```
ssh-keygen -t ed25519 -C "signing" -f ~/.ssh/id_ed25519_signing
git config --global gpg.format ssh
git config --global user.signingkey ~/.ssh/id_ed25519_signing.pub
git config --global commit.gpgsign true
git config --global tag.gpgsign true
```

15.10 Day-to-day reference

Task	Command
All CI checks locally	<code>make check</code>
Backend tests against a throwaway PostgreSQL	<code>make test-backend</code>
Frontend tests	<code>make test-frontend</code>
Apply new migrations to the running stack	<code>docker compose run --rm migrate</code>
Hot-reload frontend against the API container	<code>cd frontend && npm run dev</code>
Follow logs	<code>make logs</code>
Delete idle rate-limit buckets	<code>make prune-rate-limits</code>
Stop, keeping data	<code>docker compose down</code>
Reset everything, including the database	<code>make clean</code>

15.11 Troubleshooting

Symptom	Cause	Fix
password authentication failed for user sentinel_migrator	Database volume created by an older phase with different secrets	<code>docker compose down -v</code> , then <code>make dev</code> (local data is synthetic)
Your <code>.env</code> is missing ...	<code>.env</code> predates the current phase	Back up <code>.env</code> , run <code>make env</code> , recreate the database
Sign-in loops back to the login page	<code>__Host-</code> cookie refused on an IP address	Use <code>http://localhost:8080</code> , not <code>127.0.0.1</code>
API returns 400 for everything	Host not in <code>SENTINEL_TRUSTED_HOSTS</code>	Add it in <code>.env</code> and restart
<code>make smoke</code> reports 429s early	Your browser shares this machine's sign-in allowance	Wait two minutes and rerun
Error shows a reference ID	Expected: errors are generic	Search API logs for that <code>correlation_id</code>

CHAPTER 16

Testing and Evidence

The project's working rule is that a control without a failing-attack test is a claim, not a control. This chapter shows how the test suite is organized and what each layer proves.

16.1 The layers

Layer	Count (v0.3.0)	Runs against	Proves
Backend unit	part of 391	Pure functions	Config refusals, logging redaction, token and password primitives, egress guard, capability rules
Backend API and integration	part of 391	Real PostgreSQL (throwaway container, real roles)	Auth flows, lockout, MFA, sessions, audit chain, grants, rate limiting under concurrency
Backend security sweeps	part of 391	The live route table	Every route declares access; every role is checked; every body forbids extras; every JSON route has a response model
Frontend	45	jsdom + mocked API	Token stays in memory; refresh is single-flight; client refuses cross-origin; role-aware rendering; response validation
Smoke	34 checks	The running Docker stack through nginx	The whole user journey, end to end
Hardening	17 checks	Running containers and networks	Non-root, read-only, no capabilities, isolated database, localhost binding, headers

Backend coverage is 99%, against a CI floor of 90%. Database tests deliberately do **not** use SQLite or mocks. The grants, triggers, advisory locks and upsert semantics being tested exist only in PostgreSQL, and the test database is bootstrapped by the same role script used everywhere else.

16.2 Negative tests that matter

- **Forged tokens.** `alg: none`, algorithm substitution, wrong key, wrong audience, wrong issuer, wrong token type and expired tokens are all rejected.
- **Replay.** A rotated refresh token presented again revokes the session. A TOTP code is accepted once.
- **Enumeration.** Unknown, inactive and locked accounts give identical responses after identical work.
- **CSRF.** Cookie-bearing endpoints reject a missing or foreign `Origin` and a missing custom header.
- **BOLA.** Another user's record returns the same 404 as a non-existent one, and the attempt is audited.
- **Audit tampering.** The app role cannot update, delete or truncate. A trigger blocks even the table owner. A modified record is detected at its exact position.
- **Over-privilege.** The app role's grants must equal a hand-written matrix, so an extra grant fails.

- **Rate limiting.** Forty concurrent threads never overspend a bucket. A flood is audited once. A database outage yields `503`, never unlimited access.
- **Spoofing.** `X-Forwarded-For` from an untrusted peer is ignored. Prepend entries are skipped. Malformed hops stop the walk.
- **SSRF.** Loopback, private, link-local and metadata addresses are refused, including IPv4-mapped and 6to4 forms.
- **Error leakage.** No stack traces, paths, secrets or reflected input appear in any error.

16.3 Tests that test the tests

Several tests exist to stop the evidence going stale:

- The authorization matrix was mutated on purpose (one route weakened), and three tests failed. This proves the sweep has teeth.
- The OWASP coverage endpoint cites test names, and a test fails if any cited test no longer exists.
- The API inventory's access descriptions are compared with the independently written authorization matrix.
- The capability register's guard tests stop a SIMULATED or future-phase capability being marked as a real, implemented AWS control.

16.4 Continuous integration

GitHub Actions runs three jobs on every push and pull request: secret scanning (Gitleaks over full history); the backend (Ruff, mypy strict, Bandit, pip-audit, the role bootstrap, tests with the coverage gate, and an `alembic check schema-drift` gate); and the frontend (`npm ci --ignore-scripts`, ESLint, TypeScript, Vitest, production build, `npm audit`). The token is read-only, and every action is pinned by SHA.

CHAPTER 17

Lessons Learned So Far

A running record of what the build taught, in the order it was learned. The final, curated version becomes `LESSONS_LEARNED.md` in Phase 12.

17.1 Engineering

TESTS FIND THE BUGS CODE REVIEW MISSES

Two of Phase 2's defects were found by tests, not by reading code. Unicode digits slipped past a `\d` check and crashed a constant-time comparison. A role comparison used `is` against a string and could never match. Both were subtle, both were security-relevant, and both now have regression tests. The authorization sweep earned its place on its first run.

FRAMEWORK INTERNALS CHANGE WHAT "THE PATH" MEANS

Under nested routers, the matched route knows only its router-relative path. Rate-limit keys, metrics and audit records briefly recorded `PATCH /{user_id}` instead of the full endpoint. A startup-built map from route object to full template fixed all three consumers at once. Wherever several features need the same fact, it should be computed once.

PROXY HEADERS ARE A TRUST DECISION, SO MAKE EXACTLY ONE

Behind nginx, every client looked like the proxy, so per-IP limits would have throttled everyone together. Trusting `X-Forwarded-For` blindly would have let clients pick their own address. The answer was one middleware, outermost, trusting only a pinned network and walking right to left, with Unicorn's own proxy handling turned off so nothing else could disagree.

FAIL CLOSED, BUT FAIL CLEARLY

The first version of the limiter answered a database outage with a generic `500`. The secure outcome (refusing the request) was right, but the signal was wrong. A `503` with `Retry-After` tells clients and operators what is happening without loosening anything.

17.2 Testing and tooling

TIMING-BASED TESTS ARE MACHINE-DEPENDENT

A fixed 125-request burst tripped the probe limit in the sandbox but not on a slower workstation, because the bucket refilled during the burst. The smoke test now sends until the first `429`, and so asserts the behavior rather than an assumed speed. Similarly, React `act()` warnings that appeared only on some Node versions were fixed by waiting for the actual session-change event instead of a fixed number of ticks.

TOOLING UPGRADES CAN QUIETLY BREAK A CHECK

Docker Compose v5 changed `docker compose port` to report success for an unpublished port, which silently inverted a hardening check. The check now inspects the container's port bindings directly. A security check that can pass for the wrong reason needs its own test.

17.3 Process

ORDER THE PLAN BY COST, NOT BY NUMBER

Building every local phase before any AWS phase keeps cloud spend at zero until the infrastructure has something to run. The phase numbers stayed the same, so nothing in the specification's acceptance criteria had to be reinterpreted.

MAKE EVERY STEP RECOVERABLE

One branch per phase, milestone commits, checksummed bundles, a push after every milestone, and a signed tag per release. At any point the worst case is returning to the last tag. The owner set this as a rule before any code was written.

OPERATIONAL FRICTION IS A REQUIREMENT

A stale database volume after an upgrade produced a confusing authentication error. Instead of documenting it, the platform now diagnoses it: `make dev` detects the cause and prints the fix. Every confusing failure is a missing diagnostic.

APPENDIX A

Architecture Decision Records

All ADRs live in `docs/adr/`. Each records context, decision, security impact, alternatives considered and consequences. Implemented ADRs carry an *as implemented* addendum where reality refined the plan.

ADR	Decision	Status	Phase
0001	Private origin: internal ALB behind CloudFront VPC origins	Accepted	4–5
0002	Single origin for SPA and API; no CORS	Accepted; local via nginx	1, 5
0003	Authentication: Argon2id, 15-min JWT in memory, rotating <code>__Host-</code> refresh cookie, TOTP MFA	Implemented	2
0004	Two-layer rate limiting: WAF at the edge, PostgreSQL token buckets in the app	App layer implemented	5, 6
0005	Tamper-evident audit log: hash chain, grants, triggers, Object Lock archive	Chain, grants, triggers implemented	2, 4
0006	Amazon Bedrock as AI provider, authenticated by IAM task role	Accepted	9
0007	AI output contract (schema-bound, evidence vs inference) and human approval	Accepted	9
0008	WAF changes only through Terraform; dashboard is read-only	Accepted	5, 7, 10
0009	Provenance classification (REAL_AWS, LOCAL, SIMULATED, DEMO) enforced by tests	Implemented	1+
0010	CI identity via GitHub OIDC; SHA-pinned actions; read-only token	Pinning implemented	1, 11
0011	Security headers at both the edge and the application	App + local edge implemented	1, 5
0012	Backend layering; explicit response models; <code>extra="forbid"</code>	Implemented	1+
0013	Security gates from Phase 1 (shift left)	Implemented	1
0014	Terraform state and per-environment isolation	Accepted	3
0015	Separate database roles for migrations and runtime	Implemented	2, 4
0016	Local-first phase order to defer AWS cost	Accepted (owner decision)	—
0017	Rate-limit implementation, endpoint policy registry, client-IP trust	Implemented	6

APPENDIX B

Threat Register

Condensed from `docs/threat-model.md` (version 0.3). L×I is likelihood × impact, each from 1 to 3.

ID	STRIDE	Threat	L×I	Status
T-EDGE-01	D	Volumetric / L7 flood	2×2	Planned (P5)
T-EDGE-02	T/I	TLS downgrade, weak ciphers	1×3	HSTS in app; edge P5
T-EDGE-03	E	Origin bypass around CloudFront/WAF	2×3	Planned (P4–5)
T-EDGE-04	T	Cache poisoning / deception	1×3	Partly mitigated; edge P5
T-EDGE-05	S	Host header injection	2×2	Mitigated (P1)
T-EDGE-06	T	Clickjacking, sniffing, XSS via missing headers	2×2	Mitigated (P1)
T-ORG-01	I	Plaintext edge → origin	1×2	Planned (P4)
T-ORG-02	S	Spoofed X-Forwarded-For	2×2	Mitigated locally (P6)
T-ORG-03	T	Request smuggling / desync	1×3	Planned (P4)
T-ID-01	S	Credential stuffing / brute force	3×3	Mitigated (P2, P6); WAF P5
T-ID-02	S	Access-token theft via XSS	2×3	Mitigated (P2)
T-ID-03	S	Refresh-token replay	2×3	Mitigated (P2)
T-ID-04	T	CSRF on cookie-authenticated refresh	2×2	Mitigated (P2)
T-ID-05	I	Account enumeration	3×1	Mitigated (P2)
T-ID-06	D	Concurrent tab refresh trips reuse detection	2×1	Mitigated (P2)
T-ID-07	S	TOTP replay within window	2×3	Mitigated (P2)
T-ID-08	I	Database dump yields working second factors	1×3	Mitigated (P2)
T-ID-09	E	Default or shared bootstrap credentials	2×3	Mitigated (P2)
T-ID-10	D	Deliberate lockout of a known account	2×1	Accepted; bounded per IP (P6)
T-API-01	E	BOLA	3×3	Mitigated for users (P2)
T-API-02	E	BFLA	2×3	Mitigated (P2)
T-API-03	T	Mass assignment	2×3	Mitigated (P2, P6 sweep)
T-API-04	D	Unrestricted resource consumption	3×2	Mitigated in app (P6); WAF P5
T-API-05	I	Excessive data exposure	2×3	Mitigated (P2, P6 sweep)
T-API-07	T	Injection (SQL, command)	2×3	Mitigated in app (P2); WAF P5
T-API-08	I	SSRF via user-supplied URLs	1×3	Not exposed; guard ready (P6)
T-API-09	I	Error messages leak internals	2×2	Mitigated (P1)

ID	STRIDE	Threat	L×I	Status
T-API-10	R	Requests untraceable	2×2	Mitigated (P1)
T-API-11	T	Log injection / forging	2×2	Mitigated (P1)
T-API-12	I	Undocumented or debug endpoints	2×2	Mitigated (P1, P6)
T-API-13	D	Throttled flood fills the audit log	2×2	Mitigated (P6)
T-DB-01	I	Database exposed to the internet	1×3	Local analogue mitigated; AWS P3–4
T-DB-02	I	Database credential theft	2×3	Scoped locally (P2); Secrets Manager P4
T-DB-03	E	Over-privileged app database role	2×3	Mitigated (P2)
T-DB-04	I	Unencrypted data at rest / in transit	1×3	Planned (P4)
T-AI-01..06	LLM01/02/05/06/10	Prompt injection (direct, indirect), insecure output, data leakage, excessive agency, unbounded cost	—	Planned (P9)
T-CICD-01	E	Stolen long-lived cloud keys from CI	2×3	Planned (P11); no keys exist
T-CICD-02	T	Hijacked third-party action	1×3	Mitigated (P1)
T-SC-01	T	Vulnerable or malicious dependency	2×3	Mitigated (P1) ; SBOM P8
T-SEC-01	I	Secret committed to git	2×3	Mitigated (P1)
T-IAC-01/02	T/I	Cross-environment change; state disclosure	1×3	Planned (P3)
T-WAF-01/02	T/E	WAF weakened without review or via the app	—	Planned (P5)
T-AUD-01	R	Audit records altered or deleted	2×3	Mitigated (P2) except tail deletion (P4)
T-CNT-01	E	Container breakout / privilege escalation	1×3	Mitigated (P1)
T-INP-01	D	Non-ASCII digits crash a comparison	2×1	Mitigated (P2) ; found by tests
T-AZ-01	E	Role compared by identity, failing open or closed	2×3	Mitigated (P2) ; found by tests

APPENDIX C

Implemented Controls

Every control below is implemented and has evidence a reviewer can run or read. Planned controls are indexed in `docs/security-controls.md`.

ID	Control	Evidence
C-WEB-01	API security headers on every response, including errors	<code>test_security_headers.py</code> (200, 400, 404, 500)
C-WEB-02	SPA strict CSP, no <code>unsafe-inline</code> / <code>unsafe-eval</code>	No inline code in build output; hardening check
C-WEB-03	XSS-prone patterns banned at lint time	ESLint in CI
C-API-01	Object-level authorization (404, audited)	<code>test_users_can_read_only_their_own_record</code>
C-API-02	Function-level authorization on every route	<code>test_authz_matrix.py</code> (with mutation check)
C-API-03	Token-bucket rate limiting per IP and per account	<code>test_rate_limiting.py</code> (40-thread), make smoke
C-API-04	Unknown request fields rejected	<code>test_every_request_body_rejects_unknown_fields</code>
C-API-05	Same-origin API client; response validation	<code>client.test.ts</code>
C-API-06	Secure error envelope, no input echo	<code>test_error_handling.py</code>
C-API-07	Host header allow-list, no wildcards	<code>test_trusted_host.py</code> , <code>test_config.py</code>
C-API-08	API docs disabled when deployed	<code>test_api_docs_forced_off_when_deployed</code>
C-API-09	Endpoint policy registry equals the route table	<code>test_registry_matches_the_route_table</code>
C-API-10	API inventory with privacy-preserving metrics	<code>test_api_inventory.py</code>
C-API-11	Route-table sweeps (bodies, response models, sensitive fields)	<code>test_api_protections.py</code>
C-API-12	Outbound request (SSRF) guard	<code>test_egress.py</code>
C-ID-01	Argon2id; deployed minimum enforced; rehash on login	<code>test_security_primitives.py</code>
C-ID-02	15-min JWT, algorithm pinned, session checked per request	<code>test_forged_or_invalid_tokens_rejected</code>
C-ID-03	Rotating refresh tokens in <code>__Host-</code> cookie; reuse revokes	<code>test_auth_sessions.py</code>
C-ID-04	TOTP MFA (encrypted, no replay); hashed recovery codes	<code>test_auth_mfa.py</code>
C-ID-05	Lockout, uniform errors, constant-work dummy hash	<code>test_auth_login.py</code>
C-ID-06	CSRF: Origin allow-list plus custom header	<code>test_cross_site_requests_rejected</code>
C-ID-07	Single-use, hashed reset token in URL fragment	<code>test_auth_password_reset.py</code>
C-ID-08	No default credentials; forced setup gate	<code>test_cli.py</code> , <code>test_users_mid_setup_are_held_at_setup</code>
C-ID-09	Browser token in memory; cross-tab refresh lock	<code>session.test.ts</code> ; Chromium check
C-DB-01	Separate migrator and runtime roles; explicit grants	<code>test_database_roles.py</code>

ID	Control	Evidence
C-DB-03	Verified database TLS required when deployed	<code>test_deployed_environments_require_verified_database_tls</code>
C-AUD-01	Hash-chained, concurrency-safe, verifiable audit log	<code>test_audit_log.py</code> , <code>make verify-audit</code>
C-AUD-02	Append-only by grants and owner-proof triggers	<code>test_trigger_blocks_even_the_table_owner</code>
C-LOG-01..03	JSON logs, correlation IDs, redaction, no query strings	<code>test_logging.py</code> , <code>test_correlation_id.py</code>
C-NET-00	Database on internal network; services bound to localhost	<code>make verify-hardening</code>
C-NET-03	Client IP only via trusted proxy networks	<code>test_client_ip.py</code> , <code>make smoke</code>
C-CNT-01	Non-root, read-only, no capabilities, no-new-privileges	<code>make verify-hardening</code>
C-SEC-01..03	Secret scanning; generated secrets; per-container scoping	Gitleaks; compose review
C-CICD-01..05	SAST, SCA, least-privilege CI, coverage floor, schema drift gate	CI jobs
C-GOV-01	Provenance register with enforcement tests	<code>test_capabilities.py</code>
C-GOV-02	Code-owner review on sensitive paths	CODEOWNERS, branch protection
C-GOV-04	Exceptions with justification, compensation and expiry	EXC-0001, EXC-0002
C-GOV-05	OWASP API coverage with evidence that must exist	<code>test_owasp_coverage.py</code>

APPENDIX D

API Endpoint Inventory

All 23 endpoints at v0.3.0, generated from the endpoint policy registry. *Public*¹ means same-origin only (allowed `Origin` plus the `X-SentinelEdge-CSRF` header). *Setup* means any signed-in user, even before forced setup is complete. *Own*² means non-admins may read only their own record; others get 404 and the attempt is audited.

Endpoint	Access	Risk	Rate limit	OWASP
GET /health	Public	low	120 / min per IP	API8
GET /ready	Public	low	120 / min per IP	API4, API8
POST /auth/login	Public ¹	critical	20 / 2 min per IP	API2, API4, API6
POST /auth/mfa/verify	Public ¹	critical	20 / 2 min per IP	API2, API4
POST /auth/refresh	Public ¹ (cookie)	high	30 / min per IP	API2
POST /auth/password/forgot	Public ¹	high	5 / 15 min per IP	API2, API4, API6
POST /auth/password/reset	Public ¹	critical	5 / 15 min per IP	API2, API6
POST /auth/logout	Setup	low	60 / min per user	API2
GET /auth/me	Setup	low	60 / min per user	API3
POST /auth/mfa/enroll	Setup	high	10 / 5 min per user	API2
POST /auth/mfa/enroll/confirm	Setup	high	10 / 5 min per user	API2, API4
POST /auth/password/change	Setup	high	10 / 5 min per user	API2
GET /platform/capabilities	All roles	low	120 / min per user	API9
GET /users	ADMIN	medium	120 / min per user	API3, API5
POST /users (invite)	ADMIN	high	30 / min per user	API3, API5, API6
GET /users/{id}	ADMIN any; others own ²	medium	120 / min per user	API1, API3
PATCH /users/{id}	ADMIN	critical	30 / min per user	API1, API3, API5
POST /users/{id}/mfa/reset	ADMIN	critical	30 / min per user	API1, API5
DELETE /users/{id}	ADMIN	critical	30 / min per user	API1, API5
GET /audit-logs	ADMIN, SEC_ENG	medium	120 / min per user	API3, API5
GET /audit-logs/verify	ADMIN, SEC_ENG	medium	6 / min per user	API4, API5
GET /api-security/inventory	ADMIN, SEC_ENG, DEVELOPER	medium	120 / min per user	API5, API9
GET /api-security/owasp	ADMIN, SEC_ENG, DEVELOPER	low	120 / min per user	API5, API9

All paths are under `/api/v1`. Every route is also subject to the global ceiling of 600 requests per minute per IP. Admins cannot demote, deactivate, delete or reset MFA on themselves, and the last active admin cannot be removed.

APPENDIX E

Glossary and Edition History

E.1 Glossary

Term	Meaning
ADR	Architecture Decision Record: context, decision, security impact, alternatives, consequences
BOLA / BFLA	Broken Object / Function Level Authorization (OWASP API1 / API5)
CSP	Content Security Policy: browser-enforced allow-list of what a page may load or run
CSRF	Cross-Site Request Forgery
Fail closed	On error, refuse the action rather than allow it
Hash chain	Records each include the hash of the previous one, so any change breaks every later hash
__Host- cookie	Cookie prefix the browser accepts only with <code>Secure</code> , <code>Path=/</code> and no <code>Domain</code>
OAC	Origin Access Control: CloudFront's signed access to a private S3 bucket
OIDC	OpenID Connect: lets CI assume a cloud role without stored keys
Provenance	Whether a capability or record is <code>REAL_AWS</code> , <code>LOCAL</code> , <code>SIMULATED</code> or <code>DEMO</code>
Route template	An endpoint's full path pattern, such as <code>/api/v1/users/{user_id}</code>
SBOM	Software Bill of Materials
SSRF	Server-Side Request Forgery
STRIDE / PASTA	Threat-modeling methods: threat categories per element / risk-centric process
Token bucket	Rate-limiting algorithm: tokens refill continuously, each request spends one
TOTP	Time-based one-time password (authenticator apps)
VPC origin	CloudFront feature to reach a private, internal load balancer directly
WAF	Web Application Firewall

E.2 Edition history

Edition	Date	Release	Changes
1	October 7, 2026	v0.3.0	First edition. Phases 1, 2 and 6 as built; remaining phases as designed; reproduction guide.

Each new phase release produces a new edition. The completed phase's chapter moves from Part III to Part II and is rewritten as built, and the status tables, figures, appendices and lessons are updated.